ISSAT Sousse

Higher Institute of Applied Science & Technology of Sousse

Republic of Tunisia

Ministry of Higher Education and Scientific Research

University of Sousse

Graduation Project Report

submitted in partial fulfilment of the requirements for the

RESEARCH MASTER'S DEGREE IN COMPUTER SCIENCE

Speciality: **Intelligent Pervasive Systems**

Coding Agents Evaluation: Application to Software Architectures

Prepared by

Akram Ben Fekih

Supervised by

Mrs. Maha Khemaja

Jury

President: [jury president – to be announced]

Presented on [defence date – to be announced]

Academic Year 2025 / 2026

Topic code / ID: [topic code / ID – to be assigned]

To my family, for their unwavering support, unconditional love, and constant encouragement. You are my source of inspiration.

To my teachers, for their shared knowledge, patience, and commitment to my learning. You have guided me to success.

To my supervisors, for their expertise.

To all those who have crossed my path and contributed, directly or indirectly, to the realization of this project, I extend my sincere thanks to you.

Finally, to all those who have been part of this academic journey with me, I dedicate this work to you. Your contributions have been invaluable.

Acknowledgements

First and foremost, I would like to express my gratitude to the Almighty for granting me the courage and patience needed to complete this work.

I extend my heartfelt appreciation to my academic supervisor, Mrs. Maha Khemaja, for her guidance, patience, and unwavering support. Her critical insights have been invaluable in structuring the work and enhancing the quality of its various sections.

To the esteemed jury members, I sincerely thank you for taking the time to read and evaluate this work.

I also want to convey my deep gratitude to all my professors who have dedicated significant efforts to guide me and shape my knowledge, without whom I would not have accomplished this work.

In conclusion, I thank everyone who has contributed, directly or indirectly, to the realization of this work.

ملخص

باتت الشفرة الصادرة عن عملاء البرمجة الذاتيين تُقدَّم إلى مشاريع حقيقية، غير أنّ معايير التقييم الحالية تكاد تقتصر على الصحة الوظيفية، تاركةً جودة العمارة البرمجية دون توصيف. تقيس هذه الأطروحة مدى التزام الشفرة الصادرة عن Code Claude وCopilot GitHub وOpenAI Codex بمبادئ SOLID الخمسة، اعتمادًا على 4245 طلب دمج بلغة جافا في 1010 مستودعات، تشمل المدمجة والمرفوضة والمفتوحة.

واعتُمد لكل مبدأ مؤشرٌ بنيويٌّ حتميٌّ جرى قياس صدقه لا افتراضه: إذ خضعت 1400 حالة لحكم قراءةٍ مستقلٍّ يسجّل سبب كل حكم، مع جولة تحقّقٍ خارج العيّنة. وتراوحت الدقّة بين 330.0 و890.0، ونسبة الاسترجاع بعد ترجيح الطبقات بين 352.0 و000.1.

ويتركّز الخلل داخل الوحدة البرمجية الواحدة: فمن بين نحو 4100 مخالفة لمبدأ المسؤولية الواحدة موجودة أصلًا في الشفرة المعدَّلة، لم تُدخِل العملاء مخالفةً جديدة إلّا في اثنين إلى أربعة بالمئة من طلباتها، وفي مبدأ ليسكوف 681 مخالفة موجودة مقابل 16 مُدخَلة. كما لا تكاد الطلبات المدمجة والمرفوضة تختلف بنيويًّا، ممّا يشير إلى أنّ المراجعة البشرية لا تنتقي على أساس هذه الخصائص.

الكلمات المفتاحية: عملاء البرمجة، مبادئ SOLID، العمارة البرمجية، التحليل الساكن.

Abstract

Autonomous coding agents now author code submitted to real software projects, yet existing benchmarks measure functional correctness almost exclusively. This thesis measures whether code from Claude Code, GitHub Copilot and OpenAI Codex adheres to the five solid principles, across **4,245 Java pull requests in 1,010 repositories**, merged, rejected and still open alike.

Each principle is a deterministic structural proxy whose construct validity is measured rather than assumed: **1,400 entities** were read and labelled by an independent judge that recorded its reason for every verdict, including an out-of-sample holdout. Precision ranges from 0.330 to 0.890 and stratum-weighted recall from 0.352 to 1.000.

Risk is concentrated *inside* single units. Of roughly 4,100 single-responsibility violations already present in the code these agents touched, they introduced one in two to four per cent of their pull requests, and Liskov Substitution shows 681 violations present against 16 introduced. Merged and rejected requests are structurally indistinguishable, which indicates that human review does not select on these properties.

Keywords: coding agents, SOLID principles, software architecture, static analysis, empirical software engineering.

Résumé

Les agents de codage autonomes produisent du code soumis à de vrais projets logiciels, mais les référentiels existants mesurent presque exclusivement la correction fonctionnelle. Cette thèse mesure si le code de Claude Code, GitHub Copilot et OpenAI Codex respecte les cinq principes solid, sur **4 245 *pull requests* Java réparties sur 1 010 dépôts**, fusionnées, rejetées et encore ouvertes.

Chaque principe est un indicateur structurel déterministe dont la validité de construit est mesurée plutôt que supposée : **1 400 entités** ont été lues et annotées par un juge indépendant, avec un échantillon de contrôle hors échantillon. La précision varie de 0,330 à 0,890 et le rappel pondéré par strate de 0,352 à 1,000.

Le risque se concentre *à l'intérieur* des unités : sur environ 4 100 violations du principe de responsabilité unique déjà présentes dans le code touché, les agents n'en introduisent que dans deux à quatre pour cent de leurs demandes, et pour Liskov 681 violations sont présentes contre 16 introduites. Les demandes fusionnées et rejetées sont structurellement indiscernables.

Mots-clés : agents de codage, principes SOLID, architecture logicielle, analyse statique, génie logiciel empirique.

Contents

1 Introduction **1**

1.1 Research Context 2
1.2 Problem Statement 3
1.3 Research Objectives 5
1.4 Research Questions 6

2 Literature Review **8**

2.1 Foundations of Software Quality and Architecture 9
2.1.1 Design Principles 9
2.1.2 Code Smells 10
2.1.3 Architectural Metrics 10
2.2 Evolution and Current Capabilities of Coding Agents 11
2.2.1 First-Generation Assistants (2021–2022) 11
2.2.2 Agentic Platforms (2023–2025) 11
2.2.3 Functional Benchmark Performance 12
2.3 Evaluation of Code Quality and Architecture in AI-Generated Code 13
2.3.1 Principle Adherence and Architectural Quality 13
2.3.2 Code Smells 14
2.3.3 Repository-Scale and Human-Acceptance Evaluations 14
2.3.4 Prompting and Mitigation Studies 16
2.4 Comparative Synthesis 16
2.5 Research Gaps and Thesis Contributions 20

3 Research Methodology **22**

3.1 Research Design Overview 23
3.1.1 Research Paradigm 24
3.1.2 Study Type 24
3.2 Research Questions 24
3.3 Research Strategy 26
3.3.1 A Benchmark over Real, Merged Pull Requests 26
3.3.2 The Proxy Method: One Deterministic Evaluator per Principle . . 27
3.3.3 The Before/After Delta and the Head-State Inventory 27
3.3.4 Validation by a Reading Judge 28

3.3.5 High-Recall Screens, Suppressors, and the Validation Gate 28
3.3.6 Controlling for Repository Mix 29
3.3.7 Language Selection and the Unit-of-Analysis Precondition 29
3.4 Operationalising SOLID: Proxy-to-Principle Mapping 30
3.5 Validity 32
3.6 Ethical Considerations 33

4 Experimental Setup and Results 34
4.1 Overview 35
4.2 Dataset 35
4.2.1 Source 35
4.2.2 Curation 35
4.2.3 Reconstruction and Attrition 37
4.2.4 Resolution Quality 38
4.3 Toolchain 38
4.3.1 Selection Principles 38
4.3.2 Tools Used 39
4.3.3 Experimental Environment 41
4.3.4 Research Instruments, as Distinct from Measurement Instruments . 42
4.4 Per-Principle Proxies, Thresholds, and Equations 42
4.4.1 SRP, God-Class Rubric with Conceptual-Cohesion Suppressor . . . 43
4.4.2 OCP, Dispatch Width with Closed-Set Suppressor 45
4.4.3 LSP, Override Compatibility with False-Positive Suppressor 46
4.4.4 ISP, Interface Width and Implementer Stub Ratio 47
4.4.5 DIP, Import-Graph Delta 47
4.4.6 Component Zones, a Package-Level Descriptor, Not a Sixth Detector 48
4.5 Analysis Pipeline 49
4.6 Validation Procedure 49
4.6.1 A Second Validation Axis: Agreement with Independent Analysers 51
4.7 Results 52
4.7.1 Instrument Validation (RQ2) 52
4.7.2 Agreement with Independent Analysers 55
4.7.3 Agent Differentiation (RQ1) 57
4.7.4 Locus of Risk (RQ3) 60
4.7.5 Cross-Cutting Findings 61

5 Conclusion 64
5.1 Summary of the Work 65
5.2 Answers to the Research Questions 65
5.3 Contributions 67

5.4 Practical Implications 68
5.5 Limitations 69
5.6 Future Work 70
5.7 Closing Remark 71

Bibliography **72**

A Pipeline Run Order and Reproducibility **75**

List of Figures

1.1 Evolution of AI coding tools, from reactive autocomplete to autonomous, task-level agents. The shift from the top row to the bottom row is a change in *kind*, not degree, it is what makes evaluating these systems on function-level benchmarks alone insufficient (Section 1.2). 2

2.1 The five SOLID principles operationalised in this thesis, each mapped later to a deterministic structural proxy (Table 3.3, Section 3.4). 9

3.1 The before/after delta harness. Both trees are read via `git archive` at the merge commit and its first parent; only the *difference* the pull request introduces is charged to the agent. 27

4.1 Martin's abstractness/instability plane. The zone of pain (concrete and stable) and the zone of uselessness (abstract and unused) occupy the opposite corners; packages within ± 0.15 of $A + I = 1$ sit on the main sequence. 48

4.2 The measurement pipeline in three bands. **A** narrows AIDev to a frame balanced on agent $\times$ outcome. **B** recovers a before/after tree pair for one pull request through four methods tried in order, parses it once, and scores all five principles from the same facts; the tier reached is recorded because tier 1 is open only to merged requests. **C** validates the instrument along two independent axes, a reading judge behind an audit gate and three external analysers run over the same trees. 50

4.3 Violations present in the code the agents touched, against violations their pull requests introduced. The gap is widest for the three relation-level principles, whose violations require authoring both ends of a relation between units. 60

List of Tables

2.1 Comparative synthesis of the studies most closely related to this thesis. . . 17

3.1 Study type summary. 25
3.2 Research question summary. 26
3.3 Principle → smell → proxy → primary metric (equations in Chapter 4). . 32

4.1 Corpus composition: pull requests by agent and outcome, with repository spread and concentration. *top-1* and *top-3* give the share of a cell's pull requests contributed by its largest one and three repositories. 36
4.2 How not-merged pull requests actually ended. 37
4.3 Attrition from curated corpus to analysable corpus. 37
4.4 Analysis toolchain. 40
4.5 Machine and runtime environment. 41
4.6 Research and authoring tools. 42
4.7 What each principle asks, and what is measured instead. 44
4.8 Detector agreement with the reading judge, pooled over every judged round. TP, FP and FN are counted against the judge's verdict. Recall is *weighted*: each stratum is scaled back to its size in the corpus before the ratio is taken, with a 95% Wilson interval on the violation rate of the unflagged stratum. LSP and ISP carry 400 judged entities because each was re-drawn on a holdout. 52
4.9 Detector precision by agent, pooled over every judged round, with the number of flags each cell rests on. Precision is the quantity that matters here: an unequal *precision* across agents would manufacture a ranking that is not in the code. 53
4.10 The two refinements on the sample that produced them and on a holdout drawn with a different seed from the same population, excluding every previously judged row. 54
4.11 ISP precision by contract width, on the sample the condition was derived from. 55
4.12 Rank correlation with CK over 101,153 jointly analysed production types, with CK's internal agreement as the control. 56

4.13 Pairwise agreement on single-responsibility violations. No tool is treated as ground truth; the question is whether our rubric sits inside the spread the published tools already show. 56
4.14 Flag rate by agent. Pooled rate first, macro average over repositories in brackets. SRP is reported over classes the agent authored; the other four are reported over pull requests. 57
4.15 Flag rate by agent within bands of change size, in Java files touched. The pooled ordering holds in the largest band and does not hold in the smallest. 58
4.16 SRP flag rate, classes the agent authored against pre-existing classes it edited. 59
4.17 Flag rate by outcome on the tier-matched subset (1,521 merged and 1,177 rejected pull requests). 59
4.18 Violations present in touched code against violations introduced, after correcting both for measured precision. 60
4.19 OCP precision by dispatch width. The threshold cannot be moved to improve it. 62

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
AST	Abstract Syntax Tree
CI	Confidence Interval
CK	Chidamber–Kemerer (metrics suite)
DIP	Dependency Inversion Principle
F1	Harmonic mean of precision and recall
FP / FN	False Positive / False Negative
GRASP	General Responsibility Assignment Software Patterns
ISP	Interface Segregation Principle
LCOM	Lack of Cohesion in Methods
LLM	Large Language Model
LOC	Lines of Code
LSP	Liskov Substitution Principle
OCP	Open/Closed Principle
P / R	Precision / Recall
PMD	Programming Mistake Detector (static-analysis tool)
PR	Pull Request
RQ	Research Question
SCC	Strongly Connected Component (graph theory)
SOLID	Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
SRP	Single Responsibility Principle
TF-IDF	Term Frequency–Inverse Document Frequency
WMC	Weighted Methods per Class

Chapter

1

Introduction

1.1 Research Context

The last few years have seen a marked change in what AI tools for software development actually do. Early systems like GitHub Copilot (2021) and Amazon CodeWhisperer (2022) were, in essence, smart autocomplete, they suggested the next line or the next few lines, and the developer decided what to keep. More recent tools work differently. Claude Code (Anthropic, 2025), Cursor's agent mode, Devin (Cognition, 2024), OpenAI Codex CLI (2025), Google Jules (2025) and open-source entrants such as OpenCode and Aider are designed to operate at the level of entire tasks: they read a requirement or an issue description, plan a sequence of steps, write code, run tests, interpret failures, and revise, all without a human in the loop for each individual decision. The underlying pattern here is the ReAct paradigm (Reasoning and Acting), which interleaves natural-language reasoning with tool invocations to handle multi-step problems autonomously.

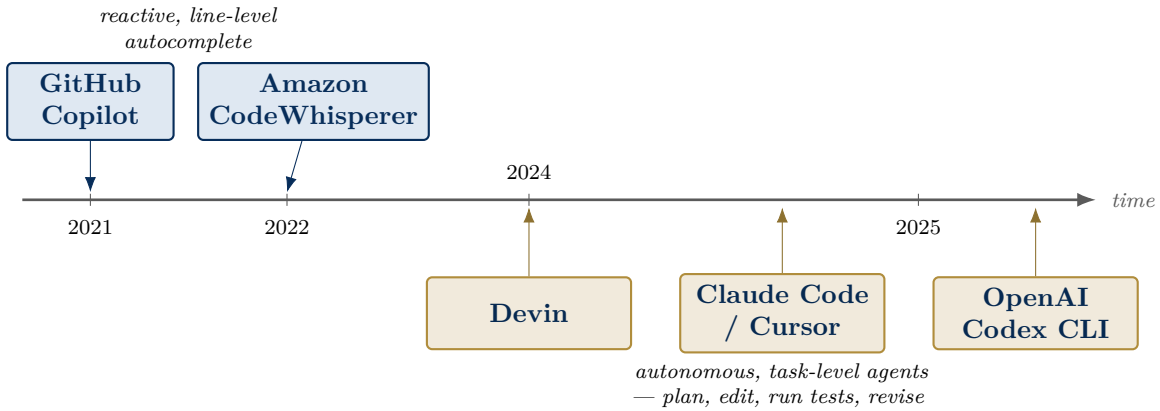


Figure 1.1: Evolution of AI coding tools, from reactive autocomplete to autonomous, task-level agents. The shift from the top row to the bottom row is a change in *kind*, not degree, it is what makes evaluating these systems on function-level benchmarks alone insufficient (Section 1.2).

In practice, this means a developer can hand off a feature description and receive a working implementation. The term *vibe coding* has emerged somewhat informally to describe this mode of collaboration, the developer maintains intent and direction, while the agent handles the implementation. A related and perhaps more concerning pattern is what Konrad et al. (2026) call *vibe architecting*: structural decisions shaped implicitly by how a prompt happens to be worded, rather than by deliberate design. Their study shows the same task with different prompts can produce systems ranging from 141 lines across two files to 827 lines across six, with no architectural decision records and no recorded rationale. That kind of structural variability is one of the things this thesis tries to characterise more precisely.

Uptake is no longer speculative, and it can be observed directly rather than through vendor surveys. The AIDev dataset (Li et al., 2025) catalogues pull requests opened

by autonomous coding agents against public repositories, and studies built on it already work at the scale of thousands: Cynthia et al. (2026a) analyse 9,427 agent-generated pull requests, and find that agent use, review effort, and merge rates all differ systematically between core and peripheral contributors. Agent-authored code is being reviewed and merged into real projects today. That is what turns its structural quality from a forecast into an empirical question, and it is the same dataset this thesis draws its corpus from.

What these figures measure, though, is speed, one narrow slice of what software quality means. The engineering discipline has spent decades developing a richer vocabulary: Frederick Brooks' analysis of accidental complexity in *The Mythical Man-Month* (Brooks, 1975), Robert C. Martin's SOLID principles (Martin, 2000), and Fowler et al.'s taxonomy of code smells in *Refactoring* (Fowler et al., 1999). The underlying point across all of this work is that passing tests and being well-designed are different things. A codebase can be functionally correct while also being tightly coupled, poorly factored, and expensive to change, and most of the cost of software comes during maintenance, not initial development.

There is growing evidence that AI-generated code raises exactly these concerns. Xu et al. (2025) find that after Copilot adoption, core developers in their study showed a 19% drop in original code productivity alongside a 6.5% increase in review burden, a measurable trade-off that productivity figures alone would obscure. A separate study by Zhu et al. (2026b) documents what they term a *Volume-Quality Inverse Law*: in AI-generated repositories, code volume predicts structural degradation almost perfectly, and neither functional test coverage nor prompt detail appears to mitigate this. That is a striking finding, if it generalises. This thesis takes it as part of its motivation, and examines whether similar patterns hold across a broader set of agents and tasks.

1.2 Problem Statement

The evaluation problem is, at its core, a mismatch: the benchmarks used to assess coding agents are designed around things that are easy to measure automatically, not around the things that actually matter when code is deployed and maintained.

On benchmark adequacy. HumanEval, MBPP, BigCodeBench (Zhuo et al., 2025), and SWE-bench (Jimenez et al., 2024) share a common structure, they ask whether a model produces code that passes a set of tests, usually at the function or single-issue level. Pass@k and coverage metrics are well-defined and reproducible, but they say nothing about whether the surrounding system is coherently structured, whether the code will be easy to modify, or whether a real reviewer would accept it. ProjDevBench (Lu et al., 2026) is one of the few attempts to evaluate end-to-end project generation, and it exposes genuine difficulties with system design, complexity optimisation, and resource

management, but across only 20 problems, which limits what can be concluded.

Separate evidence suggests the benchmark–reality gap is larger than assumed. A 2026 analysis by METR (METR, 2026) found that roughly half of solutions passing SWE-bench Verified would not be merged by the repositories' own maintainers, about a 24-point gap between benchmark score and real-world acceptance, and roughly twice the rejection rate of the human reference solutions. A different effort, SlopCodeBench (SprocketLab, 2026), finds agent code quality erodes as tasks are extended across successive checkpoints, a dimension pass@k cannot capture at all.

On structural quality. The structural evidence is worth taking seriously. Studies that prompt LLMs to detect SOLID violations directly (Pehlivan et al., 2025) report that some principles are found reliably while Dependency Inversion is consistently the hardest, and that accuracy falls as code grows more complex. Work on large AI-IDE-generated projects (Anonymous (under review), 2026) finds that functionally-working code still carries design issues such as oversized, multi-responsibility methods. Zhu et al. (2026b) characterise a recurring "machine signature" in agent-authored defects, broken abstractions, systematic duplication, over-engineering, arising from pattern mimicry rather than deadline pressure, which may make it less visible in review. What these leave open is how such deficiencies manifest at system scale, and whether individual agents differ once measured on a common corpus, the questions this thesis takes up.

The NITR benchmark (Zhu et al., 2026a) gives a sense of the depth of the structural problem: AI agents succeed on dependency-control tasks only 4.3% of the time and on responsibility-decomposition tasks only 15.2% of the time. Perhaps more troublingly, 13.3% of their AI-generated code passes all functional tests while failing architectural quality checks, what the authors call *structural oracle failure*. It suggests that test suites, as currently constructed, are not detecting something important.

On what is being evaluated. Most existing work varies the underlying model, GPT-4, Claude 3.5, Gemini Pro, while holding the agentic workflow constant. That choice assumes the scaffolding is a neutral wrapper, and the assumption has now been tested directly. Ben Sghaier et al. (2026) hold the language model fixed and vary only the *agent harness*, the middleware orchestrating prompts, tool execution and reasoning loops, across 35 sequential releases of five open-source harnesses. Evaluating Qwen Code CLI releases against a fixed task set, they trace quality fluctuations to specific development patterns and architectural components, and link individual pull requests in the harness to measured shifts in agent performance. The harness is therefore not a constant to be factored out; it is a moving part with its own measurable effect. Evaluating agents rather than models is not a terminological preference, it is the relevant unit of analysis for someone choosing between tools, and the scaffolding, context management and tool-use protocols specific

to a given agent (Copilot, Claude Code and OpenAI Codex, the three studied here) are part of what is being measured.

Taken together, these gaps point to a concrete problem: current evaluation practices leave architectural coherence, maintainability, deployability, and iterative refinement capability largely uncharacterised at repository scale. The risk is that adoption decisions rest on a narrow and favourable slice of the evidence, while the longer-term costs, inflated maintenance burden, brittle architecture, deferred refactoring, accumulate unobserved.

1.3 Research Objectives

The core question motivating this work is whether modern coding agents produce code that not only functions correctly, but also holds up against the structural and design standards that software engineering practice has developed over time. That is a harder question than it sounds, and current evaluation frameworks are not well-equipped to answer it.

More specifically, this thesis:

O1. Constructs a reproducible, repository-scale framework that measures adherence to all five **SOLID** design principles, Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion, on **real pull requests written by autonomous coding agents**, using **deterministic structural proxies** computed one principle at a time.

O2. Builds that corpus on **all three outcomes a pull request can have**, merged, rejected, and still open, rather than on merged code alone. A merged-only corpus measures what maintainers accepted, not what agents produced, and cannot distinguish the two. Including the requests that were closed unmerged makes the comparison possible.

O3. Turns construct validity into a measured quantity: each proxy is **validated against an independent reading judge** on a stratified sample of flagged and unflagged entities, reporting per-principle precision and recall rather than assuming that a structural metric captures a semantic principle. (The judge is a large language model reading the actual source; the resulting figures are rubric-versus-judge agreement, not human ground truth.)

O4. Applies the framework to **three leading coding agents**, Claude Code, GitHub Copilot, and OpenAI Codex, and characterises per-agent adherence profiles, reported both pooled and controlled for repository mix.

O5. Investigates **where code-quality risk concentrates**: inside a single unit (a class, a method) that one change can author whole, or in a relation between

units, a dependency graph, an interface, an override chain, that a single change rarely authors on both sides.

Three practical constraints are worth being explicit about. First, fully automated, from-scratch repository generation at scale would require compute and API access beyond what is available for this project; the study therefore evaluates code the agents *actually submitted* to real open-source projects, drawn from the AIDev dataset (Li et al., 2025), which is both cheaper and more ecologically valid, and which also makes each pull request a bounded unit attributable to one agent.

Second, the framework is realised for **Java**. Java is where the strongest objection to the whole design, that these principles are an awkward fit for languages that do not mandate object orientation, can be tested directly, since Java makes classes compulsory, `interface` unambiguous, `@Override` explicit, and subtyping nominal. Every SOLID principle has a syntactic home in the language, so a structural proxy is measuring the thing the principle is about rather than a distant proxy for it. The framework is language-parameterised by construction: the fact-extraction layer is the only language-specific component, and a second language can be added without changing the pipeline, the validation procedure, or the reporting.

Third, the agent set is determined by data volume rather than preference. The three agents studied are the only ones in the dataset with enough Java pull requests across all three outcomes to support a mix-controlled comparison.

1.4 Research Questions

The study is organised around three research questions. They are developed in full, with their methods of answer, in Chapter 3 (Section 3.2).

RQ1 (Adherence and differentiation)
Do modern AI coding agents differ in their adherence to the five SOLID principles on real pull requests, where do those differences concentrate, and does structural quality differ between the requests maintainers merged and the requests they turned down?

RQ2 (Measurability and construct validity)
Can SOLID adherence be measured at repository scale with deterministic, reproducible structural proxies; how faithfully do those proxies agree with an independent reading judge; and do they agree with **established, independently developed analysers** run over the same code?

RQ3 (Locus of risk)
Is code-quality risk primarily intra-unit or inter-unit, that is, does it live

inside units that a single change authors whole, or in relations between units that a single change rarely authors on both sides?

Chapter

2

Literature Review

2.1 Foundations of Software Quality and Architecture

Any evaluation of coding agents against structural quality criteria needs to be grounded in what those criteria actually are and where they come from. This section covers three layers: design principles, code smells, and architectural metrics. They are not independent, they reinforce each other in ways that matter for the analysis in later chapters.

2.1.1 Design Principles

Robert C. Martin's SOLID principles (Martin, 2000) remain the most widely cited normative framework for object-oriented design (Figure 2.1). The Single Responsibility Principle (SRP) holds that a class should have one reason to change; the Open-Closed Principle (OCP) favours extending behaviour over modifying existing code; the Liskov Substitution Principle (LSP) requires that subclasses can substitute for their parents without breaking callers; the Interface Segregation Principle (ISP) argues against broad interfaces that force clients to depend on methods they do not use; and the Dependency Inversion Principle (DIP) decouples high-level modules from low-level ones through abstraction. These are heuristics, not laws, but they connect to a long metrics literature, cohesion, coupling, and inheritance depth are all measurable proxies for the qualities SOLID targets, which is what makes automated assessment possible at all. Larman's GRASP patterns (Larman, 2004) complement SOLID at a finer grain, giving assignment-level guidance, Information Expert, Low Coupling, High Cohesion, for where responsibility should sit.

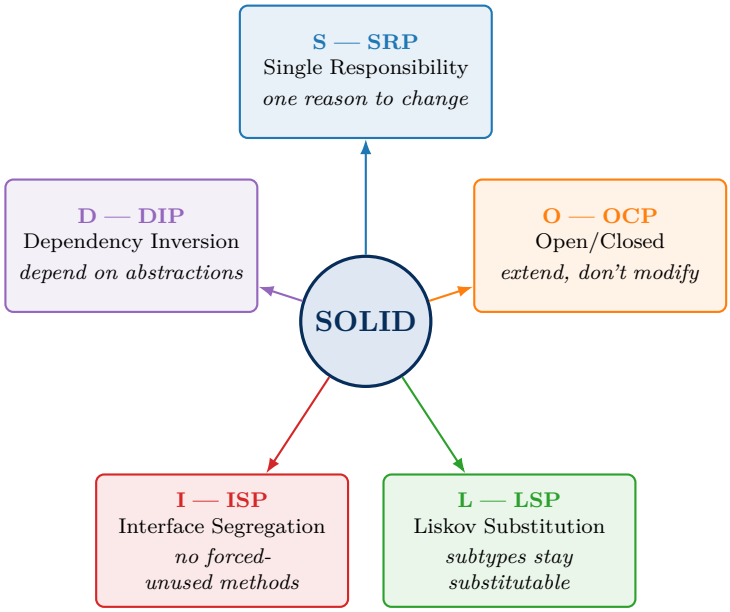


Figure 2.1: The five SOLID principles operationalised in this thesis, each mapped later to a deterministic structural proxy (Table 3.3, Section 3.4).

Evidence on how well agents follow these principles is recent and uneven. The NITR

benchmark (Zhu et al., 2026a) evaluates whether agent edits preserve maintainable structure, not just behaviour, and reports that agents handle responsibility-decomposition tasks far better than dependency-control tasks, DIP, the principle needing the most deliberate abstraction, is the hardest. It also finds cases that pass functional tests while failing structural checks, which is the more consequential point: test suites, as usually constructed, do not see these problems. A separate study prompting LLMs to detect SOLID violations directly (Pehlivan et al., 2025) reaches the same conclusion from the other side, detection accuracy is high for structurally-obvious principles but degrades on DIP and with rising code complexity. This thesis's own results (Chapter 4) independently corroborate that DIP is the least tractable of the five.

2.1.2 Code Smells

The term "code smell" was popularised by Fowler et al. (1999), which catalogued patterns, Long Method, God Class, Data Clumps, Feature Envy, Primitive Obsession, and others, that signal design problems without being outright bugs. The taxonomy's value is a shared vocabulary for what is going wrong and where. The empirical picture linking smells to defects is real but mixed rather than uniform: several large studies associate smelly classes with higher change- and fault-proneness, while others find some smells (including God Class) less fault-prone once class size is controlled for, a caution this thesis inherits, since its own God-Class proxy is deliberately size-plus-cohesion rather than size alone (Chapter 4). Automated detection tools, SonarQube, PMD, CodeScene, the CK metrics suite, operationalise the taxonomy through threshold-based rules, which is what makes cross-codebase comparison possible.

In AI-generated code, the smell picture is qualitatively different from human-generated equivalents in ways that go beyond a simple density increase. Zhu et al. (2026b) describe what they call a *Volume-Quality Inverse Law*: code volume is a near-perfect predictor of structural degradation in AI-generated repositories, and neither detailed prompting nor high functional correctness appears to interrupt the trend. A second finding from the same study, which they term a *Reasoning-Complexity Trade-off*, is counterintuitive: as models become more capable, the code they generate becomes more bloated and more coupled, not less. Their conclusion is that progress on this axis requires agents with explicit architectural oversight, which is precisely the property no functional benchmark rewards.

2.1.3 Architectural Metrics

The Chidamber–Kemerer (CK) metrics suite (Chidamber and Kemerer, 1994) provides the standard quantitative vocabulary for architectural quality: Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), Coupling

Between Objects (CBO), Response for a Class (RFC), and Lack of Cohesion in Methods (LCOM). For package-level analysis, Martin's instability and abstractness metrics (Martin, 2003) supplement CK. Of these, CBO and WMC are the primary indicators of architectural degradation, high CBO signals tight inter-class coupling; high WMC signals bloated classes, and both appear with elevated frequency in AI-generated code, as reviewed further in Section 2.3.

The RealBench benchmark (et al., 2026) operationalises architectural quality at the repository level through an Architecture@k metric on a 0–4 scale. The best current agents average around 2.25/4 (roughly 56% of ideal), and performance drops sharply as repository size grows: pass@1 falls from above 40% for repositories under 500 lines to below 15% for those exceeding 2,000 lines. That non-linear scaling failure is one of the more concrete datapoints in the literature about where architectural reasoning actually breaks down.

2.2 Evolution and Current Capabilities of Coding Agents

The agent landscape has shifted considerably over a fairly short period (Figure 1.1, p. 2). The difference between first- and second-generation tools is not just incremental capability improvement, it is a change in the kind of task the tool is designed to handle, which has direct implications for how the tools should be evaluated.

2.2.1 First-Generation Assistants (2021--2022)

GitHub Copilot, released in 2021 on top of OpenAI's Codex model, established the template: inline suggestion of the next line or function based on the surrounding context. Amazon CodeWhisperer (2022) added enterprise-oriented features, security scanning, licence attribution, while maintaining the same reactive model. At this stage, neither tool exercised autonomous initiative; they amplified what the developer was already doing. Benchmarks like HumanEval (Chen et al., 2021), which measures pass@k on 164 isolated Python problems, were a reasonable match for this capability level.

2.2.2 Agentic Platforms (2023--2025)

The shift to agentic tools began in earnest with Devin (Cognition Labs, 2024), which demonstrated end-to-end task completion, including shell execution, web browsing, and iterative debugging, without per-step human guidance. Cursor integrated deep repository indexing with agentic editing. Claude Code (Anthropic, 2025) brought a terminal-native agent with structured reasoning traces and multi-file context management. OpenCode (2025) pursued an open-source composable architecture. OpenAI's Codex CLI (2025) brought similar capabilities into a local agentic loop with shell access.

These tools differ from their predecessors in a way that matters for evaluation: they maintain state across a session, decompose specifications into sequences of actions, invoke external tools, and revise on failure. Applying HumanEval to them misses the dimension where they both succeed and fail.

Peer-reviewed comparisons of these scaffolds against each other barely exist yet, which is part of the gap this thesis addresses. What is available is engineering experience from the teams building them. Anthropic's write-up on context engineering (Anthropic, 2025) argues that an agent operates under a finite attention budget, so which tokens reach the model is itself a design decision, and names bloated or ambiguous tool sets as one of the most common failure modes in deployed agents. This is a vendor report about a vendor's own product, and Anthropic builds one of the three agents measured here, so it carries no evidential weight in this thesis; it is cited only for the design claim it makes, which the empirical chapters test rather than assume. That claim has a direct implication for evaluation: the agentic scaffolding is not a neutral wrapper around a generative model, it is an independent variable.

2.2.3 Functional Benchmark Performance

On existing functional benchmarks, second-generation agents perform well, but how well depends sharply on how the task is posed. The multi-agent AgentCoder framework (Huang et al., 2023) reaches 96.3% pass@1 on HumanEval with GPT-4, using a programmer–test-designer–test-executor loop. That result needs care: HumanEval operates at the function level, the scale at which architectural deficiencies are least visible.

Widen the task even slightly and the numbers fall. BigCodeBench (Zhuo et al., 2025) asks models to compose calls across 139 libraries under more complex instructions; across sixty evaluated models the best score reaches roughly 60%, against 97% for humans, and the authors conclude that LLMs cannot yet follow complex instructions to use function calls precisely. LiveCodeBench (Jain et al., 2024) attacks the problem from the contamination side, scoring models only on problems published after their training cut-off, and uses that holdout to expose overfitting to existing benchmarks, evidence that leaderboard standings are partly an artefact of the evaluation set rather than a property of the model.

These results point the same way: the further a benchmark moves from the isolated function, the less the headline numbers hold. SWE-EVO (Fsoft AI Center, 2025) shows how much performance changes when the task shifts to long-horizon evolution, a leading agent that resolves roughly 73% of SWE-bench Verified issues resolves only about 25% of SWE-EVO's multi-file evolution tasks (48 tasks averaging 21 changed files each).

2.3 Evaluation of Code Quality and Architecture in AI-Generated Code

There is now a growing body of work that goes beyond functional correctness to probe structural and architectural properties of agent outputs. The following sections organise this work by evaluation dimension. This thesis draws on all of it for motivation, but narrows its own empirical scope to one part of the picture, **adherence to the five SOLID principles**, measured with validated deterministic proxies, which the review below shows to be both the most design-relevant and the least reliably measured dimension at repository scale.

2.3.1 Principle Adherence and Architectural Quality

Two recent studies bear directly on SOLID adherence in agent-authored code. Pehlivan et al. (2025) prompt several LLMs to detect SOLID violations across 240 manually-validated examples and find that structurally-obvious principles (SRP, ISP) are detected reliably, that Dependency Inversion is the hardest, and that accuracy falls sharply as code complexity rises. "Beyond Functional Correctness" (Anonymous (under review), 2026) examines large projects generated by an AI IDE (Cursor), averaging roughly seventeen thousand lines across a hundred-odd files, and reports that functionally-working projects still carry design issues, including SRP-related large-method and complexity smells, that would accrue maintenance cost over time. The common thread is that passing tests and being well-structured come apart, which is the gap this thesis measures.

Konrad et al. (2026) frame the mechanism behind this. They coin *vibe architecting* for architecture shaped implicitly by how a prompt is worded rather than by deliberate design, and identify the structural decisions, framework choice, scaffolding, integration wiring, that agents make without anyone reviewing them as architecture. Their reading is that agents tend to reproduce dominant patterns from training data rather than apply design principles with intent, which predicts exactly the kind of pre-existing, unauthored structural debt this thesis observes at the pull-request level (Chapter 4).

Repository-level benchmarks add the scale dimension. RealBench (et al., 2026) evaluates repo-level generation against real requirements and UML designs across 61 repositories, and reports that architectural quality remains well below ideal and degrades as repository size grows, the same non-linear scaling failure NITR observes for maintainability. None of these studies isolates individual agents at pull-request granularity on a common corpus with validated proxies, which is the specific gap addressed here.

2.3.2 Code Smells

The most useful recent framing of AI-generated smells is qualitative rather than a single density ratio. Zhu et al. (2026b) argue that AI does not eliminate flaws so much as introduce a distinct "machine signature" of defects, recurring patterns including broken abstractions (creating an interface, then bypassing it through concrete type references), systematic duplication, over-engineering, and verbosity. The claim is not that these are simply worse than human smells but that they arise from pattern mimicry, so they are more systematic and less visible to standard review heuristics. The broken-abstraction pattern in particular is close to what this thesis measures under DIP and OCP.

This suggests a boundary that the present study takes seriously in its design. Some of what agents leave behind is *structural*, expressible as a property of the code's shape, and therefore in principle scannable. The rest is a matter of intent: code that works, and whose shape is unremarkable, but which was never written to any coherent design idea. No static rule reaches the second kind. That is the reason this thesis reports precision and recall for every proxy against an independent reading judge, instead of treating a structural flag as a verdict about design.

2.3.3 Repository-Scale and Human-Acceptance Evaluations

ProjDevBench (Lu et al., 2026) evaluates six coding agents on 20 end-to-end project-development problems across eight categories, combining online-judge testing with LLM-assisted review to score system architecture, functional correctness, and iterative refinement. The overall acceptance rate is 27.38%: agents cope with basic functionality and data structures but struggle with complex system design, complexity optimisation, and resource management. Its 20-problem corpus is too small for reliable comparison between individual agents, which is one reason the present study works over hundreds of real merged pull requests instead.

Two studies work on the same AIDev corpus this thesis draws from, and together they mark the boundary of what a mined agentic corpus has so far been used to establish. Mazloomzadeh et al. (2026) compare agentic and human pull requests longitudinally, tracking how merge rates diverge, which task categories agents are assigned, and how the characteristics of their contributions shift over successive quarters. The study is a characterisation: it establishes what agentic pull requests look like and whether they are accepted, using merge outcome and surface properties of the request rather than any measure of the resulting design.

Cynthia et al. (2026b) take the next step and measure quality directly, analysing 1,210 merged agent-generated bug-fix pull requests with SonarQube, differencing the base and merged commits so that only issues the pull request itself introduced are counted. Three of their findings bear on the present design. Apparent differences between agents

in raw issue counts largely disappear once normalised by code churn, so an agent that writes larger changes will look worse unless size is controlled. Code smells dominate over bugs, particularly at critical and major severity, which places the problem in the design register rather than the correctness register. And, most directly, **merge success does not reliably reflect post-merge code quality**. That last result is an empirical warning against treating acceptance as a quality signal, and it is tested independently here on a different quality construct and a corpus that includes the requests maintainers declined (Section 4.7).

SWE-bench (Jimenez et al., 2024) and its Verified variant remain the standard for agentic evaluation on real-world software tasks, measuring what fraction of GitHub issues an agent can resolve with all associated tests passing. The benchmark is rigorous and practically grounded, but it evaluates issue-level repair rather than system-level construction. An agent that performs well on SWE-bench may excel at targeted fixes while being poor at designing coherent architectures from a specification, the tasks are different. This thesis treats SWE-bench scores as context, not a sufficient proxy.

SWE-EVO (Fsoft AI Center, 2025) addresses long-horizon, multi-step maintenance and reports a large drop from SWE-bench Verified (∼73%) to its evolution tasks (∼25%). SlopCodeBench (SprocketLab, 2026) tracks structural degradation across iterations directly: over 20 problems and 93 checkpoints across 11 models, verbosity and structural erosion accumulate monotonically as agents extend their own prior solutions, agent code runs about 2.2× more verbose than comparable open-source projects, and no model solved any problem end-to-end, the highest checkpoint solve rate was 17.2%. Prompt-based mitigations did not stop the decay.

Sawada et al. (2026) study what happens to agent-generated code after it is merged, analysing over 1,000 files and roughly 3,200 changes across 100 popular repositories, drawn, like this thesis's corpus, from the AIDev dataset. Three findings are relevant here. AI-generated files are updated *less* frequently than human-authored ones, and the updates touch only a small fraction of file size. The modifications differ in kind: AI-authored files mostly receive feature extensions, whereas human-authored code mostly receives bug fixes. And human developers perform the large majority of the maintenance on both. The picture is therefore not one of agent code visibly failing in production, but of it being extended by people other than its author, which is exactly the situation in which structural quality, rather than test-passing, determines cost.

METR's analysis of SWE-bench Verified (METR, 2026) gives the most direct evidence that benchmark scores overstate production readiness: roughly half of agent solutions that pass the automated grader would not be merged by the repositories' own maintainers, about a 24-point gap, and roughly twice the rejection rate of the human reference solutions. Because the agents were not allowed to iterate on review feedback, this is a gap in evaluation framing rather than a fixed capability ceiling; either way, it is part of why

this thesis validates its structural proxies against a reading judge rather than trusting a passing test.

2.3.4 Prompting and Mitigation Studies

How far prompting can move design quality is still open. Evidence from the detection side (Pehlivan et al., 2025) shows prompt strategy has a large effect, an ensemble strategy helps OCP, example-based prompts help DIP, so the framing of a request clearly matters; but that says nothing about whether generation-time prompting holds up across a long task.

SlopCodeBench (SprocketLab, 2026) is discouraging on this point: quality-aware prompts improve initial quality but do not change the rate of structural degradation over successive checkpoints. Prompting addresses surface compliance without reaching the underlying architectural-reasoning problem.

Agentic linting loops, the agent writes code, runs a checker, and revises before committing, are widely recommended in practitioner accounts, but no controlled study located for this review measures their effect on design quality, as opposed to on lint-rule compliance. The gap matters for this thesis in a specific way: a loop of that kind can only enforce what a checker can express, and Chapter 4 shows that three of the five SOLID principles are not reliably expressible as a deterministic rule. An intervention that makes compliance the path of least resistance is plausible on general grounds, but its reach is bounded by the quality of the instrument in the loop, which is the thing this thesis measures.

2.4 Comparative Synthesis

Table 2.1 summarises the studies most relevant to this thesis's research questions, noting both what each establishes and where it leaves gaps.

Table 2.1: Comparative synthesis of the studies most closely related to this thesis.

Study	Agents Evaluated	Key Metrics	Principal Findings	Gaps Addressed by This Thesis
Pehlivan et al. (2025), *Are We SOLID Yet?* (Pehlivan et al., 2025)	4 LLMs (prompting)	LLM detection of SOLID violations (240 examples)	Detection strong for SRP/ISP, weakest for DIP; degrades with complexity	Detection not measurement of agent output; no per-agent PR corpus; no deterministic proxy
Beyond Functional Correctness (Anonymous (under review), 2026)	AI IDE (Cursor)	Design issues in large generated projects	Working projects still carry SRP-related smells	Single agent/IDE; no cross-agent comparison; no validated per-principle proxy
Konrad et al. (2026), *Architecture Without Architects* (Konrad et al., 2026)	Copilot, Cursor, Devin, Codex	Implicit architectural choices; "vibe architecting"	Agents reproduce patterns rather than design deliberately; same task yields 141–827 LOC over 2–6 files	Qualitative mechanism study; no per-PR quantitative proxy or validation
ProjDevBench (Lu et al., 2026)	6 coding agents	System architecture, functional correctness, iterative refinement	27.38% acceptance; agents struggle with system design, complexity, resource management	20-problem corpus; no smell/coupling metrics; no per-principle decomposition
SWE-bench (Jimenez et al., 2024)	Copilot, Devin, others	Issue-resolution rate (test pass)	Standard for issue-level agentic evaluation	Issue-level only; no system design; no smell or coupling metrics

(Table 2.1 continued)

Study	Agents Evaluated	Key Metrics	Principal Findings	Gaps Addressed by This Thesis
METR (2026) (METR, 2026)	SWE-bench Verified solutions	Maintainer merge decisions on passing PRs	∼half of test-passing agent PRs would not be merged; ∼24-point gap	Acceptance verdict only; no structural decomposition of *why*
NITR (Zhu et al., 2026a)	Multi-agent	Dependency control, responsibility decomposition, structural oracle	4.3% dependency control; 15.2% responsibility decomposition; 13.3% pass tests yet fail the structural oracle	No human baseline; no per-agent separation; synthetic edit tasks
SlopCodeBench (SprocketLab, 2026)	11 models	Verbosity, structural erosion, checkpoint solve rate	Monotonic verbosity + erosion over checkpoints; ∼2.2× more verbose than open-source; max 17.2% checkpoint solve	No agent-level separation; no SOLID/smell decomposition
Zhu, Tsantalis & Rigby (2026) (Zhu et al., 2026b)	LLM- and agent-driven projects	Code and architecture smells	"Machine signature" of defects; Volume-Quality Inverse Law; Reasoning-Complexity Trade-off	Smell-level, not principle-level; no per-agent comparison on a shared corpus
Sawada et al. (EASE 2026) (Sawada et al., 2026)	AIDev agent PRs	Post-merge maintenance effort and type	Humans do most maintenance; AI files mostly get feature extensions, human files bug fixes	Maintenance outcome only; no structural cause identified
Xu et al. (2025) (Xu et al., 2025)	Copilot users	Developer productivity, review burden	19% original-productivity drop and 6.5% more review for core devs	Organisational study, not agent evaluation; no architectural metrics

(Table 2.1 continued)

Study	Agents Evaluated	Key Metrics	Principal Findings	Gaps Addressed by This Thesis
This Thesis	**Claude Code, Copilot, Codex**	**All five SOLID principles via deterministic structural proxies, each validated (precision/recall) against a reading judge on 1,000 labelled entities; before/after PR delta**	**Agent-level SOLID adherence on 4,245 real Java PRs across 1,010 repositories, covering merged, rejected and open requests alike; risk is intra-unit (SRP, OCP) not inter-unit (LSP, ISP, DIP); merged and rejected code is structurally indistinguishable**	**Agent-level, repo-scale, per-principle, with measured construct validity rather than assumed proxy fidelity, and a corpus that includes the pull requests maintainers turned down**

2.5 Research Gaps and Thesis Contributions

Three structural gaps run through the literature reviewed above, and they collectively define what this thesis contributes.

Scale with real, attributable units. Existing structural evaluations are either functionally rigorous on small isolated tasks (HumanEval, SWE-bench) or structurally indicative on small corpora (ProjDevBench's 20 problems). This thesis works over **4,245 real Java pull requests across 1,010 repositories** from three agents, covering requests that were merged, rejected and still open alike, so that each measurement is attributable to a specific agent action rather than a synthetic task, and so that the effect of human review is measurable rather than assumed.

Depth and measured construct validity, rather than shallow breadth. Most studies evaluate one or two quality dimensions and *assert* that their metrics stand in for a design principle; a few attempt many dimensions shallowly. This thesis instead goes deep on the single most design-relevant and least-reliably-measured dimension, **SOLID adherence, all five principles**, and treats the proxy-to-principle gap as something to *measure* rather than assume: every proxy is validated against an independent reading judge, reporting per-principle precision and recall. The contribution is therefore not a wider dashboard but an honest map of **where structural measurement of SOLID works** (crisp detectors for ISP and LSP) **and where it only screens** (SRP, OCP, DIP), and of what does and does not raise that precision, and which restrictions of a structural signal raise it.

Agent-level specificity. The literature consistently conflates model capability with agent capability, varying the underlying model while treating the scaffolding as a constant. In practice, the scaffolding is not constant: Claude Code, Copilot, Cursor, Devin, OpenCode and Codex implement materially different context-management strategies, tool-use patterns, and refinement workflows. This thesis isolates those differences at the level of the agent, reporting per-agent adherence controlled for repository mix so that a single dominant repository cannot masquerade as an agent property. That the harness matters at all is no longer an assumption. Ben Sghaier et al. (2026) hold the language model constant and vary only the harness across 35 sequential releases, tracing quality fluctuations to particular development patterns and architectural components and linking individual harness pull requests to measured shifts in performance. What remains unsettled is whether that effect reaches *design* quality, and no published study measures it per principle on a shared corpus. Treating it as an open empirical question, rather than assuming either answer, is part of what this thesis contributes: Chapter 4 reports where the three agents

separate and where they do not.

Chapter

3

Research Methodology

3.1 Research Design Overview

This study uses a **quantitative empirical benchmark design** to measure how well the code produced by AI coding agents adheres to established object-oriented design principles. The unit of analysis is the **merged pull request**, a real, self-contained change an agent made to a real project, and the object of measurement is **SOLID adherence**, assessed one principle at a time with deterministic, reproducible tooling and then validated against an independent reading judge.

The study proceeds in two phases. The first, reported in Chapter 2, is a structured review of the literature on AI-assisted code generation, autonomous coding agents, and software-quality measurement, which situates the research questions and identifies the gap this thesis addresses. The second, described in this chapter and implemented in Chapter 4, is the empirical evaluation itself, carried out over a curated corpus of agent-authored pull requests drawn from the **AIDev dataset** (Li et al., 2025).

Three scoping decisions define the study and are defended below and in Section 3.5:

1. **Real pull requests rather than freshly generated repositories.** Generating new multi-agent codebases at evaluation scale is expensive in API cost and operationally complex, and it produces synthetic tasks of uncertain ecological validity. Measuring code the agents actually submitted to real projects trades experimental control for realism, and, because a pull request *is itself* a completed unit of work, it lets us attribute a measurement to a specific agent action.

2. **All three outcomes, not merged code only.** A merged pull request has passed human review, so a merged-only corpus measures the joint output of the agent and its reviewers. The corpus therefore carries **merged**, **rejected** (closed without merging) and **open** requests, balanced as far as the data allows on agent × outcome. This is what makes the review filter observable rather than invisible, and it is the single largest methodological difference between this design and a conventional mined-corpus study.

   One consequence has to be stated wherever the contrast is used: *rejected* is not *refused on quality*. Only the requests carrying an explicit maintainer objection (`changes_requested`) or a completed review that ended in closure (`reviewed_then_closed`) express a judgement about the code; the majority of closed requests are superseded, stale or abandoned. The outcome axis is therefore reported by stratum, never as a single rejected-versus-merged number.

3. **Java as the language of measurement.** SOLID is a set of object-oriented design principles, and a sceptic can reasonably argue that measuring it in a language that does not *mandate* object orientation says more about the language than about the

agent. Java removes that escape, classes are compulsory, `interface` is unambiguous, `@Override` is explicit and subtyping is nominal, so a finding that survives there is not an artefact of permissive syntax. The framework is language-parameterised (Section 3.3.7); a further language is an additional fact extractor rather than a second study.

4. **Deterministic proxies validated against a reading judge.** Every reported quality number is produced by a pure, re-runnable program; a large language model is used *only* to read the actual source and label a validation sample, never to compute a score. All agreement figures are therefore **rubric-versus-judge agreement** (Section 3.3.4, Section 4.7.1).

The design is **cross-sectional** at the level of each PR: for the four "architecture" principles it compares the repository state immediately before and immediately after the change (Section 3.3.3); for the single-responsibility principle it evaluates the touched files at the merge commit. It does not track an agent's behaviour over a long sequence of changes.

3.1.1 Research Paradigm

The study operates within a **post-positivist paradigm**: it seeks measurable, reproducible signals of design quality while accepting that no static tool fully captures a semantic property like "responsibility" or "substitutability." Each measurement is treated as a **proxy** for the underlying construct, and the gap between proxy and construct is quantified directly, through precision and recall against the reading judge, rather than merely acknowledged in prose.

3.1.2 Study Type

3.2 Research Questions

The old research questions (defect density, build/test reliability, cost-effectiveness) are retired: they described a different, multi-tool static-analysis study that was not the one carried out. The study is now organised around three questions that the built system actually answers.

RQ1, Adherence, differentiation, and outcome. *Do modern AI coding agents differ in their adherence to the five SOLID principles on real Java pull requests; where do those differences concentrate; and does structural quality differ between the requests maintainers merged and the requests they turned down?*

Table 3.1: Study type summary.

Dimension	Description
Purpose	Comparative evaluation of agents + validation of the measurement instruments
Unit of analysis	Pull request (with class- and module-level detail beneath it)
Subjects	Three AI coding agents: Claude Code, GitHub Copilot, OpenAI Codex
Outcomes	Merged, rejected (closed unmerged) and open, balanced on agent × outcome
Language	Java
Data source	AIDev dataset (Li et al., 2025), curated subset
Data collection	Automated, deterministic static-analysis pipeline over source at the PR head
Analysis	Descriptive per-agent and per-outcome rates (pooled and repository-mix-controlled), plus validation precision and recall

This is the comparative question, and it has two axes. The first is between agents: across a common, mix-controlled corpus, do agents leave measurably different structural footprints, and for which principles is there a real signal at the level of a single change? The second axis is new to this design and is the reason the corpus is built the way it is. A corpus of *merged* pull requests measures what maintainers accepted, which is not the same as what agents produced; any quality claim drawn from it is filtered through human review and is therefore survivorship-biased. Including rejected and still-open requests makes the filter itself observable: if agent-authored structure is worse in the requests that were closed unmerged, the merge decision is doing quality work, and a merged-only study would have silently inherited the benefit of that work while attributing it to the agent.

RQ2, Measurability and construct validity. *Can SOLID adherence be measured at repository scale with deterministic, reproducible structural proxies; how faithfully do those proxies agree with an independent reading judge; and do they agree with established, independently developed analysers run over the same code?*

SOLID principles are semantic; static metrics are not. RQ2 asks how far a purely structural instrument can be trusted, and answers it not with argument but with per-principle precision and recall against labels produced by a judge that reads the actual source. A second, independent axis compares the class-level instrument against three published analysers (CK, PMD and DesigniteJava) run over exactly the same source trees (Section 4.6.1). The two axes fail differently: a reading judge can be wrong in the same direction as the rubric, whereas an independently built tool cannot be wrong in *our* direction by construction.

RQ3, Locus of risk. *Is code-quality risk primarily intra-unit or inter-unit?*

RQ3 asks *where* risk lives: inside a single unit, a class, a method, that one pull request can author whole, or in a *relation between units*, a dependency graph, an interface and its implementers, an override chain, that a single pull request rarely authors on both sides. The distinction matters practically, because it predicts which kinds of design damage an agent can plausibly do in one contribution and which it cannot.

Table 3.2: Research question summary.

RQ	Focus	Core construct	Method of answer
RQ1	Agent differentiation and outcome	Per-principle SOLID adherence	Per-agent and per-outcome rates, pooled and repo-mix-controlled, on a reconstruction-matched subset
RQ2	Construct validity	Proxy fidelity	Precision and weighted recall against a reading judge on 1,400 stratified rows, including an out-of-sample holdout, plus agreement with three external analysers
RQ3	Locus of risk	Intra- versus inter-unit	Contrast between violations *present* in touched code and violations the pull request *introduced*

3.3 Research Strategy

3.3.1 A Benchmark over Real, Merged Pull Requests

The strategy is a **benchmark-style empirical study**: a fixed corpus of agent-authored changes is passed through a uniform, deterministic analysis protocol, and the results are compared across agents. Rather than commission new agent runs, the study mines completed work, which has two advantages: the code was accepted into a real project (ecological validity), and each PR is a bounded unit that can be attributed to one agent. The corresponding cost is a loss of task-level control, the study cannot hold the *difficulty* of each agent's tasks constant, which is handled through repository-mix controls (Section 3.3.6) and treated as a threat to validity (Section 3.5).

The corpus is drawn from AIDev, which records pull requests authored by autonomous agents across more than 116,000 repositories. Because the agents are wildly imbalanced in that raw data, any comparison **must sample** deliberately rather than use raw counts, or

a single agent would dominate every aggregate. The curation that produces the working corpus is described in Chapter 4 (Section 4.2).

3.3.2 The Proxy Method: One Deterministic Evaluator per Principle

Each SOLID principle is operationalised as a **pure function over source text**, a program that takes the code (as a string, or as a `{module: source}` mapping) and returns a structural measurement plus a flag, with no dependence on how the source was obtained and no external environment. This "pure evaluator" convention is what lets the single-responsibility evaluator run on individually-extracted files while the architecture evaluators reuse the same machinery over whole before/after trees, without changing the interface. Every evaluator is **deterministic**: the same input yields the same output on every run and machine, which is the precondition for a benchmark's reproducibility. The precise proxy for each principle, with equations, is given in Section 3.4 and implemented in Chapter 4.

3.3.3 The Before/After Delta and the Head-State Inventory

For the four principles whose smell is a property of the wider system, Dependency Inversion, Interface Segregation, Open/Closed, and Liskov Substitution, a single measurement of the repository "now" cannot tell us what the *agent* did. The evaluators therefore run a **delta harness**: they reconstruct the repository at the merge commit's first parent (the state just before the change) and at the merge commit (just after), analyse both, and report only the **difference** the PR introduced, a new dependency cycle, a newly-widened interface, a newly-incompatible override. This isolates the agent's contribution from pre-existing structure it merely edited (Figure 3.1).

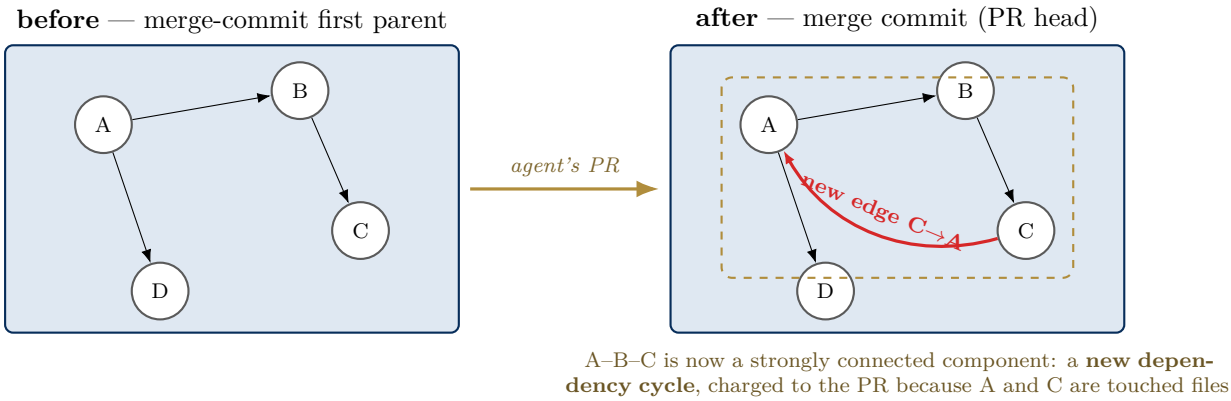


Figure 3.1: The before/after delta harness. Both trees are read via `git archive` at the merge commit and its first parent; only the *difference* the pull request introduces is charged to the agent.

Two attribution decisions follow from that aim and are stated here because they materially change what is measured. First, **authorship is separated from authorship-adjacent editing**: a class the agent created is scored separately from a class it merely touched, because scoring the touched set alone attributes the host repository's accumulated debt to the agent (Section 4.7.5, Finding 4). Second, for the relation-level principles a violation is charged when **either** side of the relation sits in a touched file, so that an agent which edits a base class or widens an interface is charged for the subclasses and implementers it breaks in files it never opened (Section 3.4).

The delta harness surfaced a result developed in Chapter 4: for ISP, LSP, and largely DIP, **single PRs almost never create the whole smell**, because a cross-unit smell spans code the PR did not fully author. To retain the signal that does exist, each of these evaluators also emits a **head-state inventory**, every interface, override pair, or dependency edge that exists at the PR head, reported as context, clearly separated from the per-PR delta.

3.3.4 Validation by a Reading Judge

A structural flag is only useful if it agrees with a considered reading of the code. The pipeline's scripts produce the flags, and a large language model (Claude) then acts as the **label judge**: for each principle a stratified sample of flagged and unflagged items is drawn, and the judge reads the *actual source*, for LSP, the overriding method and its base method side by side, and labels each as a genuine violation or not. The deterministic flag is then scored against these labels, and every precision and recall figure in this thesis is agreement between the rubric and that reading (the label column is named, e.g., `llm_srp_risk`, to keep the provenance explicit). Labelling is blind: the judge sees the entity, its metrics and its source, but not the rubric's verdict (Section 4.7.1).

One independent corroboration is worth recording: several of the strongest LSP true positives carry the original developers' *own* `# type: ignore … incompatible with supertype` comments, the very violation the deterministic rule caught, already confirmed in the repository.

3.3.5 High-Recall Screens, Suppressors, and the Validation Gate

Three of the five proxies (SRP, OCP, LSP) begin as **high-recall screens**: their thresholds are set to miss as few genuine violations as possible (recall ≈ 1.0), accepting that some flags will be false alarms. Precision is then improved, where possible, by a **suppressor** a secondary rule that removes a *nameable class* of false positives without lowering recall (the pattern is: keep the flag unless a specific, well-understood benign idiom is present). Conceptual cohesion (C3) is the single-responsibility suppressor; a closed-type-set rule is the open/closed suppressor; a four-idiom rule is the Liskov-substitution suppressor.

Every candidate change, a new threshold or a suppressor, is admitted only if it survives the same test: it must raise precision on labelled data without costing recall, and the reason it works must be nameable in advance rather than discovered by search. A restriction derived from reading false positives is cheap to justify, because the mechanism is visible in the rows it removes. A parameter found by trying values until the score improves is not, because nothing distinguishes it from fitting noise.

That distinction is why the two refinements reported in Chapter 4 are stated as restrictions of an existing signal rather than as new ones. Because both were derived by reading the false positives in a sample, scoring them on that same sample would report a fitted quantity as if it were an estimate. Each was therefore re-scored on a **holdout**: a second sample of 200 entities drawn with a different seed, excluding every row judged in any earlier round, and judged before its result was compared with the first. Section 4.7.1 reports what survived and what did not.

3.3.6 Controlling for Repository Mix

Because the corpus is mined rather than commissioned, an agent's aggregate can be distorted by *which* repositories it happens to work in, one large, messy repository can dominate an agent's pooled rate. Every comparative result is therefore reported **twice**: pooled across all of an agent's units, and **macro-averaged per repository** (the mean of per-repository rates), together with the agent's top-repository share as a concentration warning. Where the two disagree, the macro figure is treated as the more trustworthy, and ranking claims lead with it.

3.3.7 Language Selection and the Unit-of-Analysis Precondition

Three of the five proxies are **class-scoped**: the God-class rubric (SRP), the interface-width and implementer-stub rule (ISP), and the override-versus-ancestor comparison (LSP). A class-scoped instrument carries a precondition that is easy to overlook and fatal if unmet, *the corpus must actually contain classes*, and it must contain them densely enough that a per-class rate is measuring the agent rather than sampling noise.

Java satisfies that precondition by construction, and this is the main reason it is the language of the study rather than an incidental choice. A Java compilation unit must declare a type, so every pull request that touches Java source touches a class. Each of the five principles has an unambiguous syntactic home in the language: `interface` declares a contract and nothing else, `@Override` marks a substitution relationship explicitly, subtyping is nominal rather than structural, and the compiler itself rejects several of the violations that other languages permit at runtime. A structural proxy in Java is therefore measuring the construct the principle is about, rather than a distant correlate of it.

That last property produces a result in its own right, developed in Chapter 4: **what**

a compiler enforces determines which violations are expressible at all. Three of the five signals used to detect Liskov violations in a dynamically typed language cannot occur in Java code that compiles, because a changed parameter list is an overload rather than an override, parameter types are invariant, and visibility cannot narrow. The Java instrument keeps only the signals a type system cannot check. This is not a limitation of the port; it is a measurement of where each language leaves the principle unguarded.

The framework is nonetheless **language-parameterised by design**. Only the fact-extraction layer is language-specific: it turns source into a language-neutral record of types, members, supertypes, imports and member accesses, and every detector, threshold-calibration step, validation procedure and reporting path consumes that record rather than the source. A second language is therefore an additional extractor, not a second study.

3.4 Operationalising SOLID: Proxy-to-Principle Mapping

This section states, at the level of method, how each principle's semantic rule is turned into a computable proxy and why the proxy is a faithful, if imperfect, stand-in. The exact thresholds and code paths are in Chapter 4; the equations referenced here are defined there and summarised in Table 3.3.

SRP (Single Responsibility).

The forbidden smell is the *God Class*, one class doing several unrelated jobs. The proxy combines a **seven-metric size/complexity rubric** (class lines of code, method count, weighted methods, per-method complexity and parameters, and a cohesion metric, LCOM) with a **semantic cohesion signal, C3** (conceptual cohesion of classes). A class is flagged when it crosses at least two thresholds *and* its methods do not share a common vocabulary (low C3). The proxy is faithful because a God Class is characteristically both large *and* internally incoherent; combining size with cohesion is what separates "large but single-topic" (a legitimately big engine or test suite) from "large grab-bag." Its ceiling is genuinely semantic and is examined in RQ3.

OCP (Open/Closed).

The forbidden smell is the type/tag *switch statement* that must be edited for every new variant. The proxy detects every dispatch chain over one subject's type or tag (`instanceof` ladders, `getClass() ==` … comparisons, and `switch` statements over an enum or a tag value), flags those wider than a calibrated branch count, and then **suppresses** chains over a *closed* type set, one whose members are fixed by the language or by an external specification. OCP is uniquely *delta-native*: because a merged PR is itself a requirements change, the harness can watch directly whether

the agent *extended* code or *modified* a closed conditional. The closed-set suppressor encodes the actual principle, dispatching over a fixed, exhaustive set is not the open-ended extension OCP guards against.

LSP (Liskov Substitution).

The forbidden smell is an override that breaks substitutability: a an inherited method that is *refused*, a precondition strengthened by a new argument guard, or a postcondition weakened by returning `null` where the base never does. The signature faults a Liskov proxy must chase in a dynamically typed language are absent here by construction, because `javac` rejects them: a changed parameter list is an overload, parameter types are invariant, and visibility cannot narrow. The proxy therefore targets only what the compiler cannot check, and is computed over the resolved type hierarchy, which is what makes a nominal-subtyping language tractable without building each of the 1,010 repositories.

ISP (Interface Segregation).

The forbidden smell is a *fat interface* whose implementers are forced to stub methods they do not need. The proxy inventories every interface (abstract-method count = width) and every implementer (proportion of trivial stubs = stub ratio), and flags fat interfaces together with forced-stub implementers. Width alone is not the flag: a wide interface all of whose implementers use it fully is not a violation; the flag requires width plus evidence of forced stubs.

DIP (Dependency Inversion).

The forbidden smell is a high-level module depending on a concrete detail, or a dependency *cycle*. The proxy builds the internal import graph before and after the PR (via Tarjan's strongly-connected-component algorithm (Tarjan, 1972)) and flags a change that makes two modules newly mutually reachable, or that worsens the mean **Martin distance** from the main sequence beyond a threshold; it also records whether new dependencies point at abstractions or concretions. Cycle novelty admits several reasonable operationalisations that differ by an order of magnitude, so the definition is reported beside every DIP rate (Section 4.7.5, Finding 2). Validation showed the two halves are not equally trustworthy, the cycle signal is reliable, the distance signal noisy, which is itself a finding (Section 4.7).

Component zones (a descriptor, not a sixth principle).

The five proxies above all work below the package, which leaves the architectural level of the research question unmeasured. A sixth evaluator therefore places every package on Martin's abstractness/instability plane (Martin, 2003) the zone of pain (concrete and stable), the zone of uselessness (abstract and depended on by nobody), the main sequence between them, using the same A, I and D

definitions the DIP proxy already computes. It deliberately emits **no delta and no violation flag**: validation showed that the distance metric is unreliable as a signal of *change* (Section 4.7.5, Finding 2), while the *position* of a package remains a sound description of it. A package whose coordinates are undefined, no classes, or no internal coupling, is reported as unplaceable rather than as zero, since collapsing the two would file every class-free utility package in the zone of pain (Section 4.7.5, Finding 2).

Table 3.3: Principle → smell → proxy → primary metric (equations in Chapter 4).

Principle	Smell it forbids	Structural proxy	Primary metric / rule
SRP	God Class	size/complexity rubric + cohesion suppressor	LCOM; C3 conceptual cohesion
OCP	type/tag switch statement	dispatch-chain width + closed-set suppressor	branch count; `is_closed_type_set`
LSP	non-substitutable override	the three faults `javac` cannot catch, over the resolved hierarchy	refused bequest; guard count; nullable return
ISP	fat interface / forced stubs	interface width + implementer stub ratio	abstract-method count; stub ratio
DIP	concretion dependency / cycle	before/after import-graph delta	Tarjan SCC cycles; Martin distance $D = \lvert A + I - 1 \rvert$
(descriptor) component zones	*describes*, does not flag	package position on the A/I plane	zone $\in$ {pain, main sequence, uselessness, intermediate, unplaceable}

3.5 Validity

Design decisions that protect validity are stated here; the residual threats that remain after those decisions, and the boundaries of what may be concluded, are collected once in Section 5.5 rather than distributed through the results.

Internal validity. Every measurement is produced by a deterministic program over source text, so a reported number can be regenerated exactly from the corpus. Thresholds are calibrated from percentiles of the corpus's own distribution *before* any labelling, which prevents the circularity of tuning a detector on the sample used to judge it. Structural change is attributed to a pull request only through a before/after comparison of the two source trees, so a violation that already existed cannot be charged to the agent that walked past it.

Construct validity. SOLID principles are semantic and structural metrics are not, so the gap between them is measured rather than assumed. Each detector is validated on a stratified sample containing both flagged and unflagged entities, sampling only flagged entities can measure precision but can never discover a missed violation, and every label is written by a judge that reads the entity's actual source, with its reason recorded alongside the verdict. A second axis compares the class-level instrument against three independently developed analysers over the same trees (Section 4.6.1).

Statistical validity. Counts are reported beside every rate, and every rate carries a Wilson score interval, which behaves correctly for the small numerators that delta-level signals produce. Rates are reported both pooled and as a macro average over repositories, and the mix-controlled figure leads wherever the two differ, because a corpus in which one repository supplies a large share of an agent's pull requests will otherwise report that repository's habits as the agent's.

Reliability. The pipeline is resumable and idempotent, the fact extraction is cached by content hash so re-running cannot silently change an input, and each pure module carries a self-check that runs as a script. The validation material is auditable after the fact: every judged row must be traceable to the source file the judge was shown, and a mechanical audit enforces that property rather than trusting it.

3.6 Ethical Considerations

The study involves no human participants, personal data, or proprietary code. All repositories were publicly accessible and permissively licensed at the time of collection, and are cloned and redistributed unchanged only within those licences. Agent outputs were collected from the public AIDev dataset. No model training was performed; the only model used is a large language model acting as a reading judge during validation, applied to public source code.

Chapter

4

Experimental Setup and Results

4.1 Overview

This chapter describes the concrete realisation of the methodology from Chapter 3 and reports its results. It covers the curation of the corpus from the AIDev dataset (Section 4.2), the analysis toolchain and why each tool was chosen (Section 4.3), the exact per-principle proxy definitions with their thresholds and equations (Section 4.4), the analysis pipeline (Section 4.5), the validation procedure (Section 4.6), and the results, instrument validation, agent differentiation, the locus of risk, and the cross-cutting findings (Section 4.7). The limitations that bound every result are collected once, after the conclusion (Section 5.5), rather than distributed through the chapter.

The harness is written in Python and the analysis is deterministic: the third-party footprint of the core pipeline is two data libraries, the `git` command-line client, and a JDK for the fact extractor. Nothing in the measurement path depends on a network service or a model.

4.2 Dataset

4.2.1 Source

The corpus is drawn from **AIDev** (Li et al., 2025), a large collection of pull requests authored by autonomous coding agents against public GitHub repositories, enriched with repository and commit metadata. Two properties of the raw data shaped curation.

First, the released tables are not all the same population. The files named `pull_request.parquet` and `repository.parquet` contain only a curated high-visibility subset; the master tables are `all_pull_request.parquet` and `all_repository.parquet`. Reading the former while believing it to be the latter silently under-counts by an order of magnitude, and all figures reported here are taken from the master tables.

Second, per-file attribution is present for most but not all pull requests, and *which* agent is missing it changes between dataset releases. The curation therefore never branches on an agent name: where the dataset lacks a file list, the list is recovered from the GitHub API for every agent alike.

4.2.2 Curation

Curation keeps pull requests that touch **Java source by file extension**, not by the repository's declared primary language, which misses Java modules inside polyglot projects, in repositories with $\geq$**25 stars** and a licence in a permissive allowlist (MIT, Apache-2.0, BSD-2/3, ISC, 0BSD, Unlicense, CC0-1.0, MPL-2.0).

The sampling frame is then **balanced on agent $\times$ outcome**. Three agents have

enough Java across all three outcomes to support a mix-controlled comparison; the remainder are excluded on data volume rather than preference. The three outcomes are **merged**, **rejected** (closed without merging) and **open** (still awaiting a decision at collection time). Table 4.1 gives the resulting corpus.

Table 4.1: Corpus composition: pull requests by agent and outcome, with repository spread and concentration. *top-1* and *top-3* give the share of a cell's pull requests contributed by its largest one and three repositories.

Agent	Outcome	PRs	Repos	top-1	top-3
Claude Code	merged	970	271	0.9%	2.8%
Claude Code	rejected	485	171	15.1%	23.9%
Claude Code	open	283	137	7.1%	20.1%
GitHub Copilot	merged	2,544	372	7.6%	22.7%
GitHub Copilot	rejected	1,272	334	9.7%	21.4%
GitHub Copilot	open	364	149	10.2%	23.4%
OpenAI Codex	merged	630	122	4.0%	11.7%
OpenAI Codex	rejected	315	80	13.3%	36.5%
OpenAI Codex	open	93	57	17.2%	26.9%
Total		**6,956**	**1,010**		

Concentration was checked before the corpus was accepted, because a benchmark in which one repository supplies most of an agent's contributions reports that repository's house style as the agent's behaviour. No cell exceeds 17.2% in its largest repository, and every reported rate is given both pooled and as a macro average over repositories.

Rejection is not refusal. The *rejected* outcome is a coarse label, and the thesis never treats it as a quality verdict. Decomposing it by how the request actually ended (Table 4.2) shows that an explicit maintainer objection is rare: the overwhelming majority of closed requests were never reviewed at all, which in practice means superseded, stale or abandoned. Only `changes_requested` carries a recorded objection, and `reviewed_then_closed` a completed review ending in closure. Wherever the outcome axis is used, it is reported by stratum.

Table 4.2: How not-merged pull requests actually ended.

Agent	changes_requested	reviewed_then_closed	closed_unreviewed	open
Claude Code	1	11	473	283
GitHub Copilot	42	169	1,061	364
OpenAI Codex	2	36	277	93

4.2.3 Reconstruction and Attrition

Every measurement of what a pull request *changed* requires two source trees, before and after. Recovering that pair is the single most failure-prone step in the pipeline, and it is handled by four methods tried in order: the merge commit's first parent; the pull request's own commit set; the `refs/pull/N/head` reference together with its merge base; and, last, the base and head commit identifiers reported by the GitHub API.

The first method applies only to merged pull requests, which is precisely why the tier a pull request was reconstructed by must be controlled when merged and not-merged code are compared. That control is applied throughout Section 4.7. Table 4.3 accounts for every pull request between the curated corpus and the analysable one.

Table 4.3: Attrition from curated corpus to analysable corpus.

Status	PRs	Meaning
`ok`	**4,245**	analysable: valid before/after pair, Java touched
`no_touched_files`	2,540	the request touches no production Java
`repo_too_large`	150	tree exceeds the parsing budget
`archive_failed`	18	upstream decay: history rewritten or repository deleted
`no_delta_pair`	3	no tier could recover a distinct before tree
Total	**6,956**	

Two properties of that attrition are reported rather than buried, because both bound what the outcome comparison can claim. The analysable rate differs by outcome, 62.1% of merged requests, 66.9% of open, and 56.8% of rejected, so a naive merged-versus-rejected comparison is partly a comparison of *reconstructability*. And the distribution over reconstruction tiers differs sharply, since tier 1 is available only for merged requests. Both are why the outcome contrast is reported on a **tier-matched subset** (tiers 2, 3 and

4 only), where the two sides build their *before* tree the same way.

An empty file list is never accepted as a finding unless the GitHub API confirms it. An empty raw diff is a resolution failure, and treating it as "this request changed nothing" silently deletes pull requests; the pipeline records the distinction and the verifier enforces it.

4.2.4 Resolution Quality

The fact extractor resolves supertypes and imports from source alone, with no build and no classpath, so a type defined outside the repository cannot be resolved. Median supertype resolution is **87.0%** (10th percentile 37.8%, 90th percentile 98.4%), with 498 parse errors across all touched files. Resolution is even across agents, Claude Code 86.2%, Copilot 85.9%, OpenAI Codex 89.3%, so it does not bias the comparison between them.

Resolution is nonetheless bimodal by repository kind: 718 of 4,245 pull requests resolve below 50%, and these are concentrated in framework-heavy projects whose supertypes live in Spring, Android or Jakarta. The two principles that depend on resolved hierarchies, Liskov and Interface Segregation, *under*-count in those repositories. The direction matters: unresolved supertypes cause missed violations, never invented ones, so the reported rates are lower bounds there rather than inflated.

4.3 Toolchain

4.3.1 Selection Principles

Tools were chosen to be **local, free, deterministic, zero-configuration, and requiring no GPU**, the properties that make a benchmark reproducible and cheap to re-run across a thousand arbitrary repositories. In Java that rules out most of the mature options, because they analyse *bytecode* and therefore need each repository to build. ArchUnit and `jdeps` read compiled classes; SonarQube and Error Prone attach to a build; and building 1,010 arbitrary repositories at the exact commit of a pull request would mean resolving historical dependencies from the network for every one of them, which is neither reproducible nor cheap, and which fails silently in a way that looks like a clean result.

The analysis is therefore **source-only**. Facts are extracted with JavaParser and its symbol solver, which resolves types from the source tree alone, and the five evaluators are pure functions over those facts. The cost of that choice is stated rather than hidden: a supertype declared outside the repository cannot be resolved, so the median resolution rate is 87.0% and the two principles that depend on a resolved hierarchy under-count in framework-heavy projects (Section 4.2.4). The direction of that error is favourable, since an unresolved supertype removes a candidate rather than inventing one.

4.3.2 Tools Used

The core pipeline depends only on `pandas`, `pyarrow` and `git`, plus a JDK for the fact extractor and the three cross-check analysers. The large language model used as the validation **label judge** (Zheng et al., 2023) is not a pipeline tool and computes no reported metric: it reads source and produces labels, and every number in this chapter is computed by a deterministic program from those labels and the parquet outputs.

Why symbol resolution, and not a syntax-only parser. A syntax-only grammar such as tree-sitter is the cheaper option and it does not work here. tree-sitter parses Java correctly but **cannot resolve symbols**, so two classes named `Node` in different packages collapse into one identifier and every inherited relationship built on top of them is corrupted. Measured on an early build of this corpus, a syntax-only parse yielded **zero** interface contracts and only **seven** override pairs, against thousands of each once symbols were resolved. The lesson generalises beyond this study, SRP and OCP are *local* properties that syntax alone can decide, whereas LSP, ISP and DIP are *relations between types* and cannot be measured without knowing which type is which.

The decisive implementation detail is worth recording because it is invisible in documentation. A `JavaParserTypeSolver` maps package `a.b.C` to `<root>/a/b/C.java`, so it must be rooted at the **source root**, not the repository root. Rooted at the repository root, supertype resolution on one corpus repository fell from 93.3% to **17.3%**. Source roots are therefore inferred from each file's own `package` declaration, which needs no build file and handles Maven, Gradle, multi-module and flat layouts identically.

Tool coverage across the five principles is not uniform, and the gap is the contribution. Mature source-only analysers implement detectors for exactly one of the five principles. CK, PMD and DesigniteJava all carry god-class rules that speak to SRP, and DesigniteJava also carries hierarchy and cycle smells that speak partially to LSP and DIP. **No published tool implements an open/closed or interface-segregation check**, a survey of the Java static-analysis landscape returns general-purpose rule sets (coding flaws, style, security, size metrics such as `TooManyMethods`) but no dedicated detector for either principle, and the practitioner literature identifies both through manual code-smell reading rather than automation.

The reason is technical rather than accidental. Deciding whether a dispatch chain ranges over an *open* variant set requires resolving the types under test to a shared hierarchy; deciding whether an interface forces its implementers to stub requires linking a declaration to every implementation across the codebase. Both need a cross-file semantic graph, which single-file linters do not build and which compiler-backed tools obtain only from a resolvable classpath. That is precisely the capability established above on bare `git archive` trees, 87% median supertype resolution with no build, and it is what makes

Table 4.4: Analysis toolchain.

Tool	Role	Type	Used by
JavaParser + SymbolSolver 3.25.10	Java parse trees **with resolved symbols**, emitted as a language-neutral fact record	Static	all five
TF-IDF + cosine (custom)	C3 conceptual cohesion, classical information retrieval, no model	Semantic	SRP
`git` CLI	Recover the before and after source trees for a pull request	Static	OCP/LSP/ISP/DIP
GitHub REST and GraphQL API	Authoritative file lists and base/head commits where the local history is insufficient	Static	corpus construction
CK 0.7.0	Independent per-class metrics for convergent validity	Cross-check	Section 4.6.1
PMD 7.7.0	Independent design rules (`GodClass`, `TooManyMethods`, `NcssCount`, `CouplingBetweenObjects`)	Cross-check	Section 4.6.1
DesigniteJava 1.1.2	Independent design smells spanning SRP, hierarchy and cycles	Cross-check	Section 4.6.1

deterministic OCP and ISP evaluators possible here at all.

This study therefore contributes validated evaluators for the four principles the existing toolchain does not cover, and cross-checks the one it does.

4.3.3 Experimental Environment

All analysis ran on a single consumer laptop, with no cluster, no cloud compute and no GPU acceleration.

Table 4.5: Machine and runtime environment.

Component	Specification
Machine	Lenovo IdeaPad Gaming 3 (consumer laptop)
CPU	AMD Ryzen 5 5600H, 6 physical / 12 logical cores, 3.3 GHz
Memory	16 GB (15.4 GB addressable)
Storage	1 TB NVMe SSD (corpus, clones, tree and fact caches) + 512 GB NVMe system disk
Operating system	Windows 11 Home, 10.0.26200
Network	∼30 Mbit/s domestic connection
Python	3.11.9, pandas 3.0.5, pyarrow 25.0.0 (harness only)
Java	OpenJDK 17 LTS, Maven 3.9.16 (sidecar build)
Version control	Git 2.55.0

This is not incidental detail; four methodological decisions follow directly from it.

- **Source-only analysis is mandatory, not preferred.** Compiling the corpus would mean building two commits for each of 4,245 pull requests, 8,490 builds, each needing the right JDK and resolvable historical dependencies. That is beyond the available compute, and more the failures would be *biased* toward older and more complex pull requests rather than random. Every Java tool in Table 4.4 was selected because it runs on a bare `git archive` tree with no build and no classpath.

- **The per-tree fact cache exists because a cold pass costs 125 minutes** against 3.6 minutes warm, which is what makes repeated whole-corpus runs practical on this hardware.

- **The cross-check tools are serialised rather than parallel.** DesigniteJava requires a 5 GB heap on the largest repositories, which a 15.4 GB machine cannot supply concurrently, so it runs behind a semaphore while the lighter tools run in parallel.

- **Network bandwidth, not CPU, bounds the acquisition stage.** Over a 30 Mbit/s link, re-downloading the same commit for each of the delta evaluators

would dominate total runtime, which is why repositories are fetched to a shallow depth of two commits, why the before/after trees are read through `git archive` rather than a working-tree checkout, and why a gzipped per-(repository, commit) tree cache is shared by every evaluator instead of each fetching independently.

Reproducing the study needs none of this specific hardware, only enough disk for the cached corpus and a JDK, but the runtimes quoted above are the ones this machine produced.

4.3.4 Research Instruments, as Distinct from Measurement Instruments

Table 4.4 lists the software that computes reported numbers. A second, disjoint set of tools supports the *conduct and writing* of the research and touches no measurement; the two are kept separate deliberately, since only the first can affect a result.

Table 4.6: Research and authoring tools.

Logo	Tool	Description
	Zotero	Reference manager; every entry checked against its primary source before entering `references.bib`.
	Overleaf	LaTeX compilation of the thesis manuscript and the defence presentation.
	MiKTeX 25.12	Local TeX distribution. The manuscript is built with **XeTeX 4.16** under `latexmk` 4.88, since `fontspec` and the Arabic abstract require a Unicode engine rather than `pdflatex`.
	VS Code 1.133	Development environment for the pipeline and the analysis scripts.
	Git / GitHub	Version control and hosting for the pipeline, its configuration and its result tables.

GitHub occupies two unrelated positions in this study and conflating them would be an error: it hosts the pipeline's own source, and it is separately the **object of study**, since the corpus consists of merged pull requests mined from public GitHub repositories through the AIDev dataset. Nothing about the former influences the latter.

4.4 Per-Principle Proxies, Thresholds, and Equations

Each evaluator is a pure function over source. This section gives the exact computation. All thresholds live in a single configuration module and were fixed *before* the corre-

sponding validation labelling, except where a suppressor was added later and re-validated (Section 4.7.1).

None of the five principles is directly measurable. Each is a statement about *intent*, the reasons a module may change, the variants a design anticipates, the promises a subtype makes, and none of those is present in source text. What a static evaluator can decide is a structural consequence that a violation tends to leave behind. Table 4.7 states, for each principle, the gap between what the principle asks and what is actually computed, because every result in Section 4.7 is a statement about the proxy first and about the principle only through it.

Two properties of the table govern how its numbers should be read. The proxies are **conservative by construction**: each fires on a structure that a violation usually produces, never on the intent itself, so a design fault expressed without that structure is invisible to it, and recall is bounded by the choice of proxy rather than by the threshold. And the agreement figures state how often a flag matches a judge that reads the same source, they measure fidelity to *a reading of the principle*, not to ground truth.

Attribution for the relation-level principles. For LSP, ISP and DIP a violation is charged to a pull request when **either** side of the relation sits in a touched file. Charging only the flagged entity would exempt an agent that edits a base class or widens an interface and thereby breaks subclasses and implementers in files it never opened. The rule brings 81,235 override pairs and 20,705 contracts into scope across the corpus, and it changes which pull request a violation is attributed to rather than how many violations exist.

4.4.1 SRP, God-Class Rubric with Conceptual-Cohesion Suppressor

Each class receives a **rubric hit** for every threshold it crosses among seven size/complexity metrics (class lines of code, method count, weighted methods per class, maximum per-method cyclomatic complexity, maximum per-method parameter count, file lines of code, excluded from class scope, and cohesion). A class is flagged (`srp_risk`) when it accumulates $\geq$**2 hits**; a stricter operating point (`srp_risk_strict`, $\geq$3 hits) is reported alongside.

Cohesion, LCOM. Following the LCOM4 formulation, build a graph with one node per method; connect two methods if they access a shared instance attribute or one calls the other. Constructors (`__init__`/`__new__`/`__post_init__`) are excluded, because a constructor touches every attribute and would collapse an obvious three-responsibility split into a single "cohesive" component. LCOM is then the number of connected components (a saturation guard applies for classes with $\geq$4 methods). A naive computation without the constructor exclusion scored a synthetic three-responsibility God Class as fully cohesive (LCOM = 0); with the exclusion it correctly scores it as split.

Table 4.7: What each principle asks, and what is measured instead.

Principle	What the principle asks	Structural proxy actually computed	Unit	Judge agreement (P / R)
SRP	A class should have one reason to change	Seven-metric god-class rubric, class length, weighted methods, method count, per-method complexity and parameters, LCOM4, with a TF-IDF **C3 conceptual-cohesion** suppressor that spares large-but-coherent classes	Class	0.350 / 0.875
OCP	Open for extension, closed for modification	**Width of type- and tag-dispatch chains** (`instanceof` ladders, `switch` on an enum or tag), flagged past a calibrated branch count, with a **closed-set suppressor** for dispatch over a fixed, exhaustive set	Dispatch chain	0.330 / 0.733
LSP	A subtype must be substitutable for its base	**Override-versus-ancestor comparison** over the resolved hierarchy, restricted to the signals a compiler cannot check: refused bequest, strengthened precondition, weakened postcondition	Override pair	0.420 / 0.352 (**1.000** on 64 flags restricted to refused bequest)
ISP	Clients should not depend on members they do not use	**Contract width and implementer stub ratio**, every interface and abstract class inventoried, with the share of members each implementer leaves unimplemented	Contract	0.335 / 0.552 (**0.786** at $\geq$30 members)
DIP	Depend on abstractions, not concretions	**Newly cyclic module pairs** in the import graph, charged to the pull request only when one end is a file it changed	Pull request	**0.890 / 1.000**
Zones	(descriptive) Martin's A/I position	Zone of pain, main sequence, zone of uselessness, intermediate	Package	Not validated, **descriptive only** (Section 4.4.6)

† The Liskov figure is for the flag as originally composed; restricting it to refused bequest raises precision to 1.000 (Section 4.7.1). ‡ The DIP flag reports the new-cycle signal alone. The distance-from-main-sequence signal was measured, found to be right 0 times out of 7, and excluded from the flag (Section 4.4.6).

LCOM is reported as **undefined** when the method graph admits no edge *by construction*. A type whose methods touch no field and call nothing has no edges for reasons of syntax alone, and its normalised value pins near 1.0 regardless of design, mean LCOM is 0.857 for interfaces against 0.425 for everything else. Scoring such types would let the cohesion knob fire as a second rubric hit on types whose only real signal is width, double-counting the method-count knob. The condition is tested on the *graph* rather than on the declaration keyword, so an interface whose default methods call one another keeps its value.

Conceptual cohesion, C3 (the semantic suppressor). Treat each non-constructor method as a bag of identifier and docstring terms (Marcus et al., 2008), form a smoothed **TF-IDF** vector per method, and take the mean pairwise cosine similarity:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} \tag{4.1}$$

$$C3 = \operatorname*{mean}_{i<j} \cos(v_i, v_j) \tag{4.2}$$

where v_i is the TF-IDF vector of method i. C3 ranges from 0 (methods share no vocabulary, mixed responsibilities) to ∼1 (all methods discuss the same domain, cohesive). A rubric-flagged class whose $C3 \geq 0.4$ is **un-flagged**: its methods demonstrably share one vocabulary, so it is the "large but single-topic" class that a pure size proxy cannot forgive. This is the C3-suppressor pattern that recurs, in structural form, for OCP and LSP.

4.4.2 OCP, Dispatch Width with Closed-Set Suppressor

The evaluator finds every **dispatch chain**, an `if/else if` ladder or a `switch` that discriminates on one subject, and classifies each by what it tests: `instanceof` and `type_eq` (comparison against a `.class` literal or `getClass()`) are type-based; `value_switch` and `switch_tag` are tag-based. The corpus contains **41,880** in-scope chains of at least two branches, 7,069 type-based and 34,811 tag-based.

Thresholds were calibrated from the observed branch-count distribution *before* any label existed. Type-based chains are flagged at **≥3 branches**, which is also Martin's minimum "switch" and sits at the 62nd percentile of type chains; tag-based chains at **≥8**, the 86th percentile, a deliberately conservative bar because short value ladders are usually parsers or state machines rather than missing polymorphism.

The **closed-set suppressor** encodes the real open-versus-closed distinction:

$$\text{smell} \;=\; (\text{branch_count} \geq \text{threshold}) \;\wedge\; \neg\, \text{is_closed_set}(\text{subject})$$

where `is_closed_set` holds when the subject resolves to an `enum`. Java makes this cheap and exact: an enum's variant set is fixed by the language, so a switch over one cannot be extended by a client, and dispatching over it is not the open-ended growth OCP guards against. 6,073 chains are suppressed on that rule.

The limitation of the rule is that it recognises only the *language's* notion of a closed set. The closed sets that actually recur in this corpus are external specifications, JDBC type codes, protobuf scalar types, Unicode blocks, HTTP status families, which are closed in fact but open to the compiler. Section 5.6 names recognising them as a concrete improvement.

4.4.3 LSP, Override Compatibility with False-Positive Suppressor

What a compiler already guarantees is not worth detecting. In a dynamically typed language a Liskov proxy must check that the override kept the base's signature, because nothing else will. In Java `javac` enforces that: a method with a different parameter list is an *overload* rather than an override, parameter types are invariant so they cannot be narrowed, return types may only be made more specific, and visibility cannot be reduced. Code that compiles cannot contain those faults, so a detector that looks for them in Java finds nothing and merely inflates its own denominator. The proxy is aimed instead at what the compiler cannot see: whether the override still *behaves* like the base.

For every internal subclass→nearest-ancestor override pair the evaluator computes three signals:

- `refused_bequest`, the base supplies a working implementation and the override replaces it with a body that only throws `UnsupportedOperationException` or `NotImplementedException`;
- `precondition_strengthened`, the override adds argument guards that throw where the base accepted the call;
- `postcondition_weakened`, the override can return `null` on a path where the base never does.

Three scope cuts keep the denominator honest. Constructors and static methods are excluded, since neither is a substitutable call site. The five `Object` methods (`equals`, `hashCode`, `toString`, `clone`, `finalize`) are excluded, because every class declares them and they would swamp the signal. Most importantly the base must supply a **real inherited body**: a pair whose ancestor is an abstract or interface declaration is not an LSP relation at all, since there is no bequest to refuse and no precondition to strengthen, and counting those pairs would add every interface method in the corpus to the denominator.

Section 4.7.1 shows that the three signals are not equally trustworthy, and that restricting the flag to refused bequest is the single most effective change made to any detector in this study.

4.4.4 ISP, Interface Width and Implementer Stub Ratio

A **contract** in Java is an `interface` or an `abstract class`, and the evaluator indexes both over the whole repository, following the resolved hierarchy so that members inherited from a super-interface are counted as part of the contract they impose. The corpus holds **20,705** contracts, 16,562 interfaces and 4,143 abstract classes.

Two scope cuts are applied before anything is measured, and skipping either turns the result into an artefact. A contract must declare **at least one abstract member**, and it must have **at least one implementer inside the repository**. Without the second cut every empty marker interface counts as a contract nobody was forced to implement.

For each contract the evaluator records its **width**, the number of abstract members, and for each implementer a **stub ratio**, the share of the members it supplies whose body is empty or throws a refusal exception. A contract is flagged at **≥13 members**, the 86th percentile of observed widths, calibrated before labelling; 2,944 contracts (14.2%) exceed it, and 744 implementer relationships carry a forced stub. Width alone is a screen rather than a verdict: an implementer that uses the whole contract is not a violation however wide the contract is, which is why the stub ratio is recorded beside the width and why Section 4.7.1 returns to what width alone can and cannot support.

4.4.5 DIP, Import-Graph Delta

The evaluator builds the internal import graph before and after the PR, restricted to modules the PR touched, and computes:

- **Cycles**, strongly-connected components of size >1 (Tarjan's algorithm (Tarjan, 1972)). Novelty is decided on the **whole** import graph and expressed as **newly mutually-reachable module pairs**: (u, v) is newly cyclic when the two modules became mutually reachable across the change, and the PR is flagged when such a pair involves a touched file. Three operationalisations of "a new cycle" were implemented and they differ by an order of magnitude, so the definition is reported beside every DIP rate (Section 4.7.5, Finding 2). Attribution is at the **compilation-unit** level rather than the package level. Package-level filtering is too coarse for the common single-package layout: measured on one Android project it admitted 44 newly cyclic pairs where only one involved a file the pull request had touched.
- **Martin metrics** per package, Instability $I = C_e/(C_a + C_e)$, Abstractness $A = N_a/N_c$, and distance from the main sequence $D = |A + I - 1|$, where C_e/C_a are

efferent/afferent couplings and N_a/N_c are abstract/total class counts. A PR is flagged if it worsens mean D by more than 0.15.

- **Abstract-dependency ratio**, the share of a touched module's new dependencies that land on modules defining an abstraction, reported as context.

4.4.6 Component Zones, a Package-Level Descriptor, Not a Sixth Detector

The five proxies above operate below the package: a class, a dispatch chain, an override pair, an interface, a module edge. A design claim about *architecture* needs at least one measurement at the level where architecture is normally discussed, so a sixth evaluator classifies where each package sits on Martin's abstractness/instability plane (Figure 4.1). It reuses the same `martin_metrics` implementation the DIP evaluator already shares across languages, so the coordinates are the ones defined above:

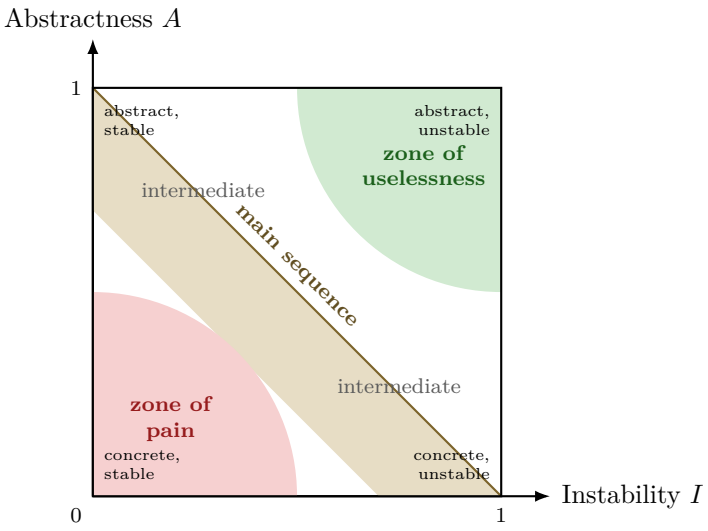


Figure 4.1: Martin's abstractness/instability plane. The zone of pain (concrete and stable) and the zone of uselessness (abstract and unused) occupy the opposite corners; packages within ± 0.15 of $A + I = 1$ sit on the main sequence.

- $I = C_e/(C_e + C_a)$, instability, 0 when nothing depends outward.
- $A = N_a/N_c$, abstractness, the share of a package's classes that are abstract.
- Two corner regions of radius **0.5**, the **zone of pain** near $(A{=}0, I{=}0)$, concrete and stable, so everything depends on it and it cannot change; and the **zone of uselessness** near $(1, 1)$, abstract and depended on by nobody. Packages within $\pm$**0.15** of the line $A+I = 1$ are **on the main sequence**; the rest are **intermediate**. Both the radius and the band are presentation choices rather than measurements, and are stated here so a reader can reposition them.

- **Unknown** is a fourth outcome, kept deliberately distinct from zero: A is undefined for a package that declares no classes and I is undefined for one with no internal coupling at all. Collapsing either to 0 would place every class-free utility package in the zone of pain, manufacturing the study's headline result out of an encoding error.

Three scoping decisions bound what this evaluator claims. First, it reads the **after** tree only and emits **no delta, no violation flag and no agent ranking**, it describes the codebases the agents worked in, and it is not validated against a judge. That restriction is empirical rather than cautious. The distance-from-main-sequence signal was measured as a change signal and was correct 0 times out of 7, because mean D moves when the graph is *recomposed*, files added or moved, and not when a dependency actually inverts. Distance from the main sequence is a discredited *change* signal on this corpus; the A/I *position* of a package is a different and defensible use of the same coordinates. Calling a zone-of-pain package a violation would require its own validation round.

Second, packages are aggregated on the **parent** package rather than on the first dotted segment of the module name. The top-segment rule that the cycle detector uses is correct for attribution but useless here, because a reverse-DNS prefix places most of a repository in one package, which gives $C_e = C_a = 0$ and therefore an undefined I. Attribution is otherwise identical to the DIP harness.

Third, the classifier is pure and self-checking, including the assertion that an undefined coordinate never resolves to a zone.

4.5 Analysis Pipeline

The pipeline is a sequence of independent, resumable stages, each reading upstream Parquet outputs and writing its own, so any stage can be re-run in isolation (Figure 4.2).

The four architecture evaluators each reconstruct the before/after trees independently, fetching the merge commit and its first parent and reading the tree via `git archive`. Every stage is resumable: each pull request is processed independently, permanent outcomes are retained while transient failures are retried, and every attempt is recorded so that attrition is auditable rather than silent.

4.6 Validation Procedure

For each principle a **stratified sample** is drawn, deliberately balancing flagged and unflagged items, and, for the entity-level principles, stratifying across the three agents. The **label judge** (Claude) then reads the actual source of each item and records a binary label (`llm_<principle>_risk`); for LSP the extractor presents the overriding method and

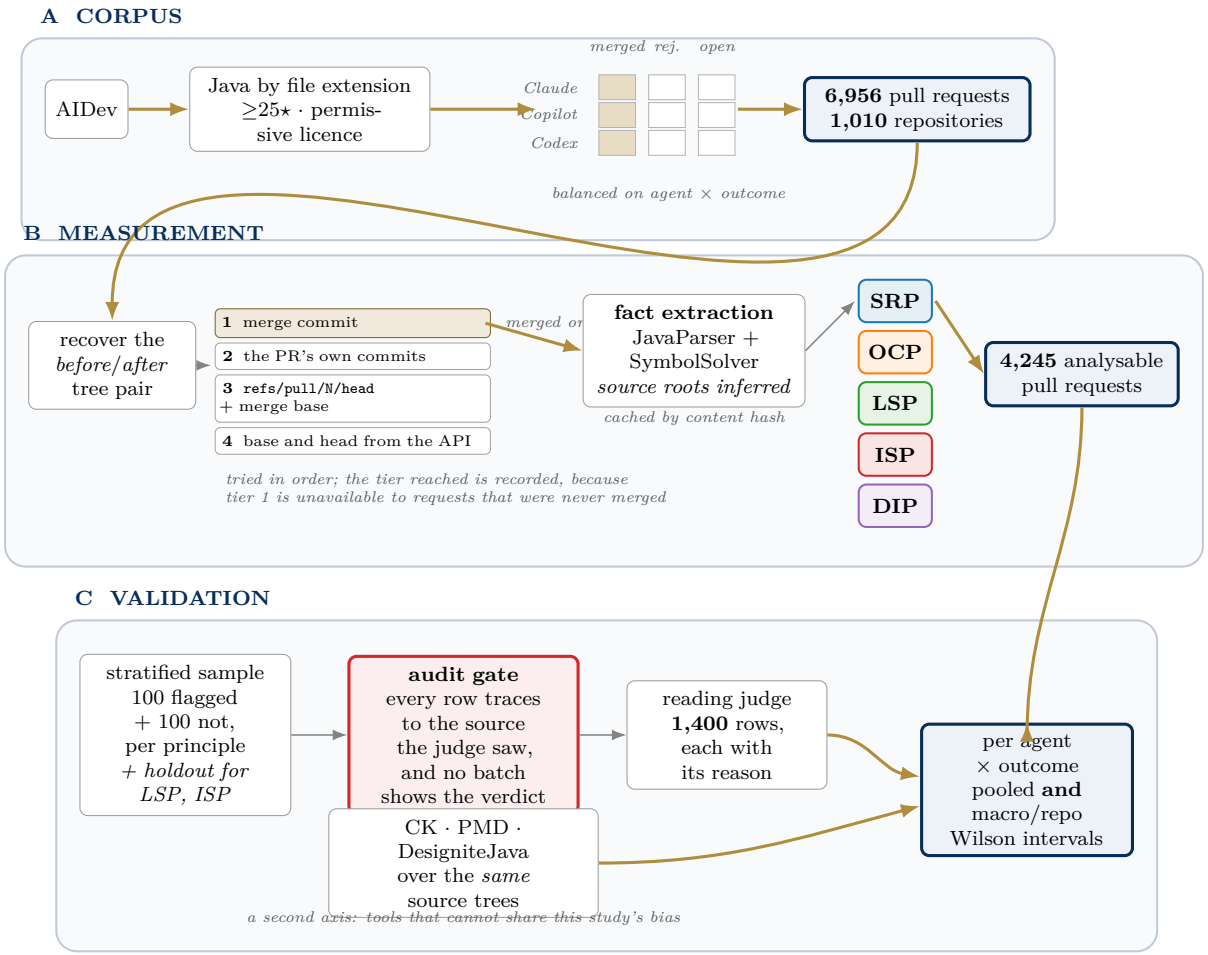


Figure 4.2: The measurement pipeline in three bands. **A** narrows AIDev to a frame balanced on agent × outcome. **B** recovers a before/after tree pair for one pull request through four methods tried in order, parses it once, and scores all five principles from the same facts; the tier reached is recorded because tier 1 is open only to merged requests. **C** validates the instrument along two independent axes, a reading judge behind an audit gate and three external analysers run over the same trees.

its base method side by side, since substitutability cannot be judged from one side alone. The deterministic flag is scored against the label as precision, recall, and F1, overall and per agent.

Each principle is validated on **one** stratified sample of 200 entities, drawn with a seed recorded alongside the result. The sample deliberately balances flagged and unflagged items, because a sample drawn only from flagged items can measure precision but can never discover a violation the detector missed.

Three of the five detectors were shipped with thresholds set from corpus percentiles before any label existed, so their agreement figures were never fitted to the labels that score them. Two quantities were fitted: the restriction of the Liskov flag to refused bequest, and the interface width condition, both read off the false positives in the first sample. Scoring a fitted quantity on the sample that produced it reports optimism, not

performance.

Both were therefore re-scored on a **holdout**: a further 200 entities per principle, drawn with a different seed from the same production population, excluding every row judged in any earlier round, extracted through the same gate and judged under the same rubric before the two rounds were compared. Once a detector has been changed in response to a sample, that sample can no longer estimate the changed detector, which is the whole reason the exclusion has to cover every prior round rather than only the most recent one.

4.6.1 A Second Validation Axis: Agreement with Independent Analysers

Judge-based validation has a structural weakness that no amount of sampling fixes. The judge reads the same code the rubric read, and can therefore be wrong *in the same direction* as the rubric, if both treat "large" as "does too much," neither will notice. A second axis is added in Java, where mature source-only analysers exist: three independently developed tools are run over **exactly the same source trees** the detector sees, and their verdicts are compared with ours.

The comparison uses one head-state tree per repository, taken from the `after` tree of that repository's most recent pull request, across all 1,010 repositories in the corpus. CK 0.7.0 supplies per-class metrics, PMD 7.7.0 supplies design rules, and DesigniteJava 1.1.2 supplies design smells. All three run at their **default thresholds**, deliberately: re-tuning them until they agreed with us would destroy the only property they are being asked for.

Three design points matter for interpretation. First, agreement is reported **symmetrically**, with no tool treated as ground truth. The question is not "is our rubric right?" but "does our rubric sit inside the spread that published detectors already show among themselves?", which requires measuring the established tools against *each other* as well as against ours.

Second, CK's `lcom` is LCOM1 and its `lcom*` is Henderson-Sellers (Henderson-Sellers, 1996), whereas ours is LCOM4. These are different formulas, so cohesion is compared by **rank correlation only** and never as an agreement check.

Third, all three tools fail quietly, and every failure mode flatters the study: a crashed analysis produces an empty result file that reads as "this repository is clean". Each tool's per-repository status is therefore recorded, and repositories where a tool failed are excluded from that tool's comparison rather than counted as agreement.

One property of rank-based convergent validity bounds its interpretation. **Spearman correlation is invariant to any monotone rescaling**, so a high correlation on a size metric establishes agreement on *ordering* only. It cannot certify the scale on which either tool measures, and a high correlation on lines of code is entirely consistent with two tools counting lines differently.

4.7 Results

4.7.1 Instrument Validation (RQ2)

Each detector was validated on a stratified sample of 200 entities: 100 that it flagged and 100 that it did not. Sampling only flagged entities can measure precision but cannot discover a missed violation, so both strata are drawn. The sample is taken after the same production filter the report uses, so the validated population and the reported population are the same one.

A judge read the source of each entity and recorded a verdict together with the reason for it. The material shown to the judge omits the detector's own answer, so agreement cannot be inflated by anchoring, and every judged row is required to trace back to the file the judge was shown. Table 4.8 gives the result over all 1,000 rows.

Table 4.8: Detector agreement with the reading judge, pooled over every judged round. TP, FP and FN are counted against the judge's verdict. Recall is *weighted*: each stratum is scaled back to its size in the corpus before the ratio is taken, with a 95% Wilson interval on the violation rate of the unflagged stratum. LSP and ISP carry 400 judged entities because each was re-drawn on a holdout.

Principle	judged	TP	FP	FN	Precision	Recall (weighted)
DIP	200	89	11	0	**0.890**	1.000 (0.410–1.000)
LSP	400	84	116	3	0.420	0.352 (0.159–0.614)
SRP	200	35	65	1	0.350	0.875 (0.562–0.975)
ISP	400	67	133	9	0.335	0.552 (0.400–0.700)
OCP	200	33	67	2	0.330	0.733 (0.440–0.909)

Why recall must be weighted, and what that costs. Each sample is half flagged and half unflagged, but the corpus is not. For Liskov the corpus holds 1,546 flagged override pairs against 79,689 unflagged ones, so a sampled unflagged row stands for roughly fifty times as many entities as a sampled flagged one. Counting the two strata as though they were equally common would put Liskov recall at 0.966; scaling each stratum back to its true size puts it at **0.352**. The correction matters most exactly where the flagged population is smallest, and it is the reason the intervals in Table 4.8 are wide: a weighted recall rests on the handful of violations found in the unflagged stratum, three for Liskov across 200 rows, and that handful is multiplied by a large factor.

The consequence is a real limit on what the flag counts mean. For SRP, OCP and DIP the detectors do miss few violations and over-report many, and their flag rates can be read as upper bounds on what the judge would recognise. For LSP and ISP they cannot: those two detectors see the violations that announce themselves structurally and are blind to the rest. The three Liskov violations found in the unflagged stratum say

what the blind spot is made of. A Vaadin root node rejects every non-null parent with an `IllegalStateException` where its base accepts any; a CommCare screen controller overrides a method that disables a login button and shows an error with an *empty body*; an nd4j operation implements `setY` as `this.y = null`, a setter that discards its argument. None throws where the detector looks, and one does nothing at all. Structural proxies catch refusals that are declared and miss refusals that are silent.

Precision below 0.5 for four principles is the substantive finding of RQ2 rather than a defect to be apologised for. A structural proxy answers a syntactic question; the principles are semantic; and the gap between the two is what these numbers quantify. The remainder of this section identifies where the gap comes from, and in two cases closes it.

Evenness across agents. A detector that over-flags one agent more than another manufactures a difference that is not in the code, so a per-agent ranking is only admissible if precision is comparable across agents. Table 4.9 gives precision by agent.

Table 4.9: Detector precision by agent, pooled over every judged round, with the number of flags each cell rests on. Precision is the quantity that matters here: an unequal *precision* across agents would manufacture a ranking that is not in the code.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
DIP	0.778 (28/36)	0.943 (50/53)	1.000 (11/11)
LSP	0.500 (53/106)	0.324 (22/68)	0.346 (9/26)
ISP	0.422 (35/83)	0.280 (26/93)	0.250 (6/24)
SRP	0.327 (18/55)	0.263 (10/38)	1.000 (7/7)
OCP	0.338 (23/68)	0.280 (7/25)	0.429 (3/7)

The OpenAI Codex column rests on 7 to 26 flags per principle, so its cells are unstable and both values of 1.000 stand on 11 and 7 flags respectively. The comparison the agent ranking actually needs is between Claude Code and Copilot, which carry 25 to 106 flags each. On those two the picture is consistent rather than alarming: Claude Code's precision is equal or higher on every principle, which means the detectors are, if anything, *less* likely to over-flag it than Copilot, and the higher flag rates reported for Claude Code in Section 4.7.3 are therefore not an artefact of the instrument. Codex's *rates* are computed over its full 770 pull requests and are unaffected; it is only its per-agent precision that is thin.

Two refinements earned from the labels. Two detectors can be made substantially more precise by a change that the labels themselves identify, and in both cases the change is a restriction rather than a new signal.

The Liskov detector originally combined three signals. Decomposed against the labels, they behave very differently: *refused bequest*, a base method that does real work, replaced

by an override whose body is a bare `throw new UnsupportedOperationException`, was correct on 31 of 31 flags, while *postcondition weakened* was correct on 6 of 36 and *precondition strengthened* on 3 of 33. Restricting the flag to refused bequest gives precision 1.000 (95% interval 0.943–1.000) over both validation rounds, on 64 flags, at recall 0.736 within the judged sample. An earlier, independent round gave 51 of 62 for the same signal and the same near-zero rates for the other two, so the restriction rests on three separate samples rather than one.

The Interface Segregation detector improves under a width condition. Precision rises monotonically with contract width and the transition is sharp (Table 4.11). Requiring at least 30 members raises precision from 0.320 to 0.862 (0.694–0.945) on the sample the condition was read from.

What the holdout did to both. Each refinement was then scored on 200 entities it had never seen (Table 4.10). They behaved in opposite ways, and the difference is instructive rather than incidental.

Table 4.10: The two refinements on the sample that produced them and on a holdout drawn with a different seed from the same population, excluding every previously judged row.

Refinement	Fitted sample	Holdout	Pooled
LSP, restricted to refused bequest	1.000 (31/31)	**1.000** (33/33)	1.000 (64/64)
ISP, restricted to ≥30 members	0.862 (25/29)	**0.704** (19/27)	0.786 (44/56)

The Liskov restriction held exactly, without a single false positive in 64 flags across two independent samples. The interface width condition lost sixteen points, and its fitted value of 0.862 lies outside the holdout's own interval of 0.515 to 0.841. The reason is visible in the form of the two rules. Refused bequest is *categorical*: a method body that is nothing but a throw either is or is not that shape, and there is no quantity to tune. A width of 30 members is a *numeric threshold chosen by looking at where the false positives stopped*, and a threshold picked that way absorbs some of the noise of the sample it was picked on. Both refinements are real, and the pooled figures are the ones to quote; but only one of them was free of the optimism that fitting introduces, and the holdout is what separates them.

Judge consistency. 174 entities appear in both validation rounds, mostly because the Dependency Inversion detector flags only 195 pull requests in the whole corpus, so a second sample of 100 necessarily repeats earlier rows. The round-one labels were not consulted until the second round was complete. Agreement between the two independent readings is **89.7%** (156 of 174), ranging from 84.0% for ISP to 92.2% for DIP. This is a

Table 4.11: ISP precision by contract width, on the sample the condition was derived from.

Members	Flags judged	True	Precision
0–17	43	5	0.116
18–29	28	2	0.071
30–49	14	11	0.786
50–99	10	9	0.900
≥100	5	5	1.000

measure of the judge, not of the detectors, and it places a ceiling on how precisely any agreement figure in Table 4.8 can be known.

4.7.2 Agreement with Independent Analysers

The judge-based figures in Section 4.7.1 share a structural weakness: the judge reads the same code the detector read, and can be wrong in the same direction. The second axis removes that possibility by construction. CK 0.7.0, PMD 7.7.0 and DesigniteJava 1.1.2 were run at their own default thresholds over the head-state tree of a seeded random sample of 150 repositories, giving **111,029 production types** in common with our own analysis. None of the three tools was written for this study, so none can agree with it for the wrong reason.

Size and complexity converge; cohesion does not, and neither does CK with itself. Rank correlation against CK is near-total on the size and complexity metrics and weak on cohesion (Table 4.12). The weak result is not evidence that our cohesion metric is wrong, because **CK's own two cohesion variants correlate 0.490 with each other**, and one of them correlates 0.240 with its third. Our LCOM4 agrees with CK's LCOM1 about as well as CK's measures agree among themselves. Cohesion is an unreliable metric family, which is why it is used here only as one hit in a seven-metric rubric and never on its own.

81.1% of the classes we flag sit inside CK's own top decile by weighted methods, and 79.7% inside its top decile by lines of code, so the flags land where an independent size measure also points.

The decisive comparison: the published tools agree with each other less than either agrees with us. PMD's `GodClass` rule and DesigniteJava's *Insufficient Modularization* and *Multifaceted Abstraction* smells both target what SRP targets, so the three detectors can be compared directly on the same 82,553 production types.

The two established tools agree with each other at $\kappa = 0.384$, less than either agrees with the rubric built here. **4,304 of our 6,326 flags (68.0%) are corroborated by**

Table 4.12: Rank correlation with CK over 101,153 jointly analysed production types, with CK's internal agreement as the control.

Comparison	Spearman ρ	Reading
ours ~ CK weighted methods	**0.977**	converges
ours ~ CK method count	**0.969**	converges
ours ~ CK lines of code	**0.944**	converges
ours ~ CK response for class	0.789	converges
ours ~ CK `lcom` (LCOM1)	0.288	weak
ours ~ CK `lcom*`	0.203	weak
control: CK against itself		
CK `lcom` ~ CK `lcom*`	0.490	CK's own two variants
CK `lcom` ~ CK `tcc`	0.388	
CK `lcom*` ~ CK `tcc`	0.240	

Table 4.13: Pairwise agreement on single-responsibility violations. No tool is treated as ground truth; the question is whether our rubric sits inside the spread the published tools already show.

Pair	Cohen's κ	Precision	Recall	Flag rate
ours ~ PMD `GodClass`	**0.574**	0.435	0.940	ours 7.7%
ours ~ DesigniteJava	**0.531**	0.494	0.651	PMD 3.5%
PMD ~ DesigniteJava	**0.384**	0.543	0.331	Designite 5.8%

at least one independent tool, and 1,573 by both. Against PMD's broader test of any two design rules firing together, agreement reaches $\kappa = 0.701$.

This does not establish that the rubric is correct. It establishes that its disagreements with published detectors are no larger than theirs with each other, which is the strongest claim convergent validity can support and the one thing a judge reading the same source could never provide.

Two divergences, both explained by unit rather than by verdict. Agreement on substitutability is **zero** ($\kappa = 0.000$). The cause is definitional: DesigniteJava's *Rebellious Hierarchy* charges the *supertype* whose contract is being broken, while the detector here charges the concrete *subclass* that refuses the inherited method. The two tools name different classes for the same underlying fault, so class-level agreement is near zero by construction rather than by disagreement.

Dependency inversion divides on granularity. Compared at the compilation-unit level the two agree at $\kappa = 0.389$; aggregated to packages, which is DesigniteJava's unit, agreement falls to $\kappa = 0.080$ while recall rises to 0.924. Our detector finds nearly everything Designite finds and charges it to a finer unit. Open/Closed and Interface Segregation have no counterpart in any of the three tools and are validated by the reading judge alone.

Coverage. DesigniteJava produced results for 141 of the 150 repositories, CK joined to 92.1% of our types, and PMD failed on 252 files. Every failure is recorded and those types are excluded from that tool's comparison rather than counted as agreement, because all three tools fail by producing an empty result, which would otherwise read as a clean bill of health.

4.7.3 Agent Differentiation (RQ1)

Table 4.14 gives the flag rate for each principle by agent. The macro average over repositories is given beside the pooled rate throughout, and is the figure to read where the two disagree, because it prevents a single large repository from reporting its own conventions as an agent's behaviour.

Table 4.14: Flag rate by agent. Pooled rate first, macro average over repositories in brackets. SRP is reported over classes the agent authored; the other four are reported over pull requests.

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP (authored classes)	9.5% (8.4%)	**5.1%** (3.5%)	6.8% (7.4%)
OCP	10.6% (10.1%)	6.4% (3.0%)	**2.1%** (2.2%)
ISP	8.7% (6.0%)	6.7% (4.0%)	**2.9%** (5.0%)
DIP	6.5% (5.1%)	4.1% (2.8%)	**3.2%** (5.4%)
LSP	0.7% (0.3%)	0.3% (0.1%)	**0.0%** (0.0%)

Claude Code has the highest pooled rate on all five principles and Codex the lowest on four. The ordering is consistent, but three qualifications bound what may be read into it.

First, the agents did not work in the same period. The median pull request date is August 2025 for Codex, January 2026 for Copilot and March 2026 for Claude Code. An agent measured on older code is measured against different repositories, different framework versions and a different population of tasks, and this study cannot separate the agent from its era.

Second, the macro and pooled figures disagree for Codex on ISP and DIP, where its macro rate exceeds Copilot's while its pooled rate is lower. That pattern means Codex's flagged pull requests are spread across many repositories while its unflagged volume is concentrated in few, and the mix-controlled figure is the one to quote.

Third, Codex contributes 770 analysable pull requests against 2,366 for Copilot, so its intervals are wider throughout.

Change size confounds the comparison, and partly explains it. Cynthia et al. (2026b) report that apparent differences between agents in raw issue counts largely disappear once normalised by code churn, because an agent that writes larger changes accu-

mulates more issues without writing worse code. The same confound is present here and has to be reported, because it qualifies the ordering above.

The three agents do not write changes of the same size. The mean number of Java files touched is 46.4 for Claude Code, 18.5 for Copilot and 10.9 for OpenAI Codex, and Codex puts 63.0% of its pull requests in the smallest band against roughly 49% for the other two. Flag rates rise steeply with size for every agent and every principle, which is expected: a delta-level detector has more opportunity to fire when more code changes.

Table 4.15: Flag rate by agent within bands of change size, in Java files touched. The pooled ordering holds in the largest band and does not hold in the smallest.

Principle	Files touched	Claude Code	Copilot	OpenAI Codex
OCP	1–3	1.3%	**1.9%**	0.4%
	4–10	**7.0%**	3.5%	1.9%
	11+	**31.3%**	19.3%	8.9%
ISP	1–3	0.4%	**0.9%**	0.6%
	4–10	**7.7%**	6.0%	4.3%
	11+	**24.7%**	19.6%	9.7%
DIP	1–3	0.4%	0.5%	**0.8%**
	4–10	1.8%	2.4%	**3.7%**
	11+	**22.3%**	13.7%	12.1%

Table 4.15 splits the comparison by change size, and the result is not a simple confirmation of the pooled ordering. In the largest band the ordering holds on all three principles, and the separation is wide: Claude Code flags 31.3% of its largest Open/Closed changes against 8.9% for Codex. In the smallest band the ordering does not hold. Copilot is highest on Open/Closed and Interface Segregation among changes touching three files or fewer, and Codex is highest on Dependency Inversion in both of the two smaller bands, reversing its pooled position.

Two conclusions follow, and the second is the one to carry forward. First, part of the pooled difference between agents is a difference in how much they change at once rather than in how they design. Second, the pooled ordering is not purely an artefact, because it survives within the band where most of the flagged pull requests are. The defensible statement is therefore narrower than the pooled table suggests: **on large changes Claude Code introduces more structural violations than Copilot, and Copilot more than Codex; on small changes the agents are not distinguishable and the ordering sometimes inverts.**

The Liskov row deserves separate comment. Codex introduced no refused-bequest violation in 770 pull requests, and the corpus-wide rate is 16 pull requests out of 4,245. This is not a null result produced by a weak detector: the same detector finds 681 refused-bequest relationships present in the code these agents touched. The rate is low because

the violation requires authoring a base and an override that contradict each other, which a single pull request rarely does.

Authored code against edited code. The Single Responsibility detector is the only one that operates per class rather than per pull request, which makes it the only principle where agent-authored code can be compared directly against the code the agent merely edited.

Table 4.16: SRP flag rate, classes the agent authored against pre-existing classes it edited.

	Authored	**Edited**	**All touched**
Pooled	7.7%	13.5%	12.2%
Macro over repositories	5.6%	20.9%	18.2%
Classes	13,712	48,433	62,145

Classes the agents wrote are flagged at roughly half the rate of the pre-existing classes they edited, and the gap widens under repository-mix control. The comparison is between two populations rather than two treatments, so it does not establish that agents write better classes than the humans who preceded them: new classes are smaller and younger, and a class accumulates responsibilities over years of maintenance. What it does establish is that the flagged classes in this corpus are overwhelmingly not the agents' work.

Merged code against rejected code. The outcome axis is the reason the corpus includes requests that were never merged. Because the reconstruction tier available to a pull request depends on whether it was merged, the comparison is made only within tiers 2, 3 and 4, where both sides recover their *before* tree the same way.

Table 4.17: Flag rate by outcome on the tier-matched subset (1,521 merged and 1,177 rejected pull requests).

Principle	**Merged**	**Rejected**	**Difference**
OCP	8.3%	8.0%	−0.4 pp
ISP	7.5%	6.6%	−0.9 pp
DIP	4.6%	6.6%	+2.0 pp
LSP	0.5%	0.5%	±0.0 pp

There is no consistent structural difference between the pull requests maintainers merged and those they closed. Three of the four differences are under one percentage point, and the largest, Dependency Inversion at two points, runs in the direction of rejected code carrying more violations, on a base rate small enough that the intervals overlap.

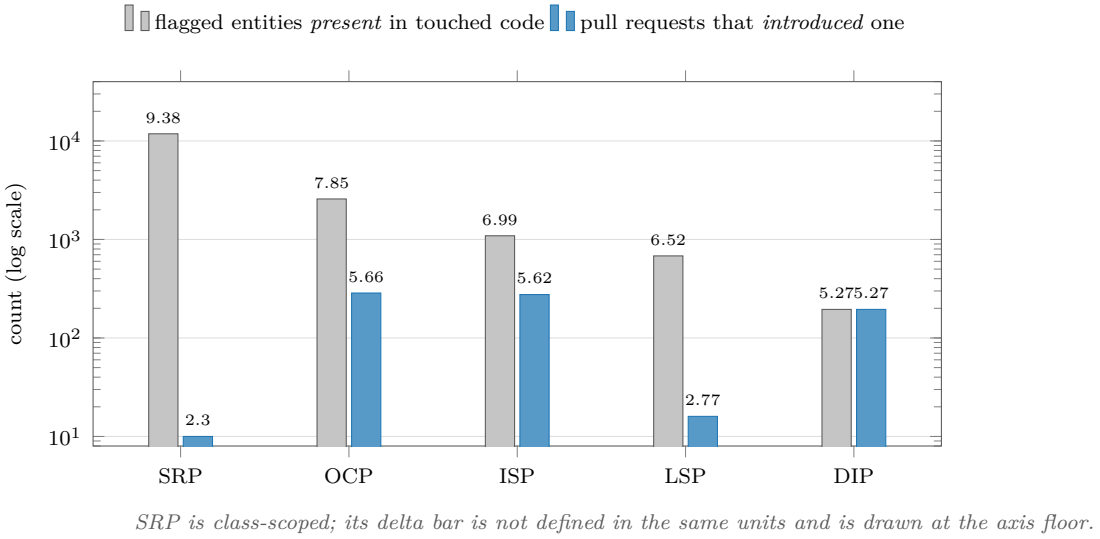
The reading is not that review is ineffective. It is that SOLID adherence, as measured here, is not what review is deciding on. Review attends to correctness, scope, tests and fit with the project's direction; the structural properties measured here are close to invisible in that process. This also settles a methodological question for anyone building on merged corpora: for these signals, merged code is not a biased sample of agent output, so a merged-only corpus would have reached the same conclusions. That could not have been known without measuring the requests that were turned down.

4.7.4 Locus of Risk (RQ3)

The detectors report two quantities that must not be conflated. A violation *present* in the code a pull request touched may have been written by anyone at any time. A violation the pull request *introduced* is one that the before/after comparison attributes to this change. Table 4.18 places them side by side, with the precision figures from Table 4.8 applied so that both columns estimate genuine violations rather than flags.

Table 4.18: Violations present in touched code against violations introduced, after correcting both for measured precision.

Principle	Flagged entities present	Est. genuine	PRs that introduced one
SRP	11,822 classes	≈4,100	n/a
OCP	2,577 dispatch chains	≈850	286 (2.2% corrected)
ISP	1,090 contracts	≈350	276 (2.1% corrected)
LSP	681 override pairs	≈270	16 (0.4% corrected)
DIP	195 cycles	≈175	195 (4.1% corrected)



SRP is class-scoped; its delta bar is not defined in the same units and is drawn at the axis floor.

Figure 4.3: Violations present in the code the agents touched, against violations their pull requests introduced. The gap is widest for the three relation-level principles, whose violations require authoring both ends of a relation between units.

Between two and four per cent of pull requests introduce a SOLID violation, in codebases that already contain thousands. The three inter-unit principles show the effect most sharply: 681 refused-bequest relationships exist in the touched code and pull requests introduced 16 of them.

The explanation is structural rather than behavioural. Single Responsibility and Open/Closed are properties of *one* unit, a class, a dispatch chain, which a single pull request can author whole. Liskov, Interface Segregation and Dependency Inversion are properties of a *relation between* units: a base and its override, an interface and its implementers, a cycle among modules. Authoring both ends of such a relation in one contribution is rare, so these violations accumulate across many changes and across authors rather than arriving in one.

The practical consequence is that per-change review, whether human or automated, can plausibly catch intra-unit damage and structurally cannot catch inter-unit damage. Detecting the latter requires a view of the repository over time, which no single diff provides.

4.7.5 Cross-Cutting Findings

Finding 1: what a compiler enforces determines which violations are expressible. Three of the five signals used to detect Liskov violations in a dynamically typed language cannot occur in Java code that compiles. A changed parameter list produces an overload rather than an override; parameter types are invariant; visibility cannot narrow and checked exceptions cannot broaden. Covariant return types are legal and safe. The Java detector therefore keeps only the signals a type system cannot check, and the labels confirm which of those survive contact with real code: refused bequest is correct on 31 of 31 flags, while the two behavioural-contract signals are correct on 6 of 36 and 3 of 33. A language's type system does not merely make violations harder; it removes some of them from the space of things a program can express.

Finding 2: structural width is a valid signal for one principle and not for another, for a principled reason. Interface width predicts Interface Segregation violations sharply (Table 4.11): precision is 0.071 below 30 members and 0.900 above 50. Branch width predicts Open/Closed violations not at all, staying between 0.270 and 0.400 across every band (Table 4.19).

The asymmetry follows from what the two principles are about. Interface Segregation is a statement about size: "clients should not be forced to depend on members they do not use" is violated by breadth, so a width threshold measures the construct directly. Open/Closed is a statement about whether a set of variants can grow. A 64-branch switch over PostgreSQL type identifiers is closed, because the set is fixed by a wire protocol; a

Table 4.19: OCP precision by dispatch width. The threshold cannot be moved to improve it.

Branches	Flags judged	True	Precision
0–5	37	10	0.270
6–9	23	9	0.391
10–15	19	6	0.316
16–30	10	4	0.400
≥31	11	4	0.364

five-branch switch over supported storage backends is open, because the next release adds one. Counting branches cannot separate them, and no threshold placement will.

Finding 3: the closed sets that matter are external specifications. The Open/Closed detector carries a suppressor for dispatch over a closed type set, written to recognise sets fixed by the language. It fired on none of the 100 flagged rows in the validation sample. Reading the false positives shows why: the closed sets that occur in practice are fixed by specifications outside the language, JDBC type codes, protobuf scalar types, FHIR binding strengths, Unicode blocks, AWT cursor constants, Android key codes, and by generated parser tables, where a 356-branch keyword switch is machine-produced from a grammar file that is the actual source of truth. This gives a concrete and testable route to a more precise detector, which is that the suppressor should recognise external specification enums rather than only language builtins.

Finding 4: the risk is intra-unit. Stated as a single sentence for the abstract: of roughly 4,100 single-responsibility violations and 270 substitutability violations present in the Java code these agents touched, the agents introduced a violation in two to four per cent of their pull requests, and almost all of what the instruments find is pre-existing human code. The five principles agree on this, and the reason they agree is the intra-unit versus inter-unit distinction developed in Section 4.7.4.

Finding 5: agent-authored classes are flagged less often than the classes they edit. 7.7% against 13.5% pooled, and 5.6% against 20.9% under repository-mix control (Table 4.16). The comparison is observational and the two populations differ in age and size, so this is a description of the corpus rather than a claim that agents write better classes than people. It does mean that a reader who assumes agent code is the source of the structural problems in these repositories has the direction wrong.

Finding 6: review does not select on these properties. Merged and rejected pull requests are structurally indistinguishable on four principles once reconstruction method

is held constant (Table 4.17). Since human review is the only quality filter these contributions pass through, and it does not appear to act on SOLID adherence, the case for an automated structural check at review time rests on it catching something review demonstrably does not.

Chapter

5

Conclusion

5.1 Summary of the Work

This thesis set out to answer a question that functional benchmarks are not built to answer: does the code that autonomous coding agents write for real projects hold up against the structural design standards, the SOLID principles, that software engineering practice has developed over five decades? Chapter 1 motivated the question from the gap between benchmark scores and production acceptance. Chapter 2 showed that the existing literature either measures structural quality shallowly across many dimensions, or deeply on corpora too small to separate agents. Chapter 3 built a measurement design around that gap: one deterministic proxy per principle, validated against an independent reading judge rather than assumed to be faithful. Chapter 4 realised that design over **4,245 analysable Java pull requests across 1,010 repositories**, written by three agents, and covering merged, rejected and still-open requests alike.

Two design decisions distinguish the study from a conventional mined-corpus analysis. The corpus is not restricted to merged code, so the effect of human review is observable rather than baked into the sample. And construct validity is measured rather than argued: 1,000 flagged and unflagged entities were read and labelled, with the reason for each verdict recorded, so that every rate in Chapter 4 arrives with a known error profile.

The remainder of this chapter answers the three research questions (Section 5.2), states what the thesis contributes (Section 5.3), gives a practical reading for anyone choosing between agents or building on one (Section 5.4), states the limitations that bound every claim (Section 5.5), and closes with the directions the work leaves open (Section 5.6).

5.2 Answers to the Research Questions

RQ1, Do agents differ, where, and does review select on it? The agents differ, but the difference is narrower than a pooled comparison suggests. Claude Code carries the highest pooled flag rate on all five principles and OpenAI Codex the lowest on four, including no refused-bequest violation in 770 pull requests (Table 4.14).

That ordering is partly a difference in how much the agents change at once. Mean change size ranges from 46.4 Java files for Claude Code to 10.9 for Codex, and flag rates rise steeply with size for every agent, exactly the confound Cynthia et al. (2026b) identify when they normalise by churn. Splitting the comparison by change size (Table 4.15) shows the ordering holding in the largest band, where Claude Code flags 31.3% of its Open/Closed changes against 8.9% for Codex, and failing in the smallest, where Copilot leads on two principles and Codex on Dependency Inversion. The defensible claim is therefore conditional: **on large changes the agents separate in the order given; on small changes they do not separate at all.**

Three further qualifications bound even that. The agents worked in different periods,

with median pull request dates seven months apart, so agent and era cannot be separated here. The pooled and mix-controlled figures disagree for Codex on two principles, and the mix-controlled figure is the one to read. And Codex contributes a third as many analysable pull requests as Copilot.

The outcome axis returns a cleaner answer. Once reconstruction method is held constant, pull requests that maintainers merged and pull requests they closed are structurally indistinguishable: three of four differences are under one percentage point, and the largest runs in the direction of rejected code carrying *more* violations (Table 4.17). Human review, on this evidence, is not selecting on SOLID adherence. That agrees with Cynthia et al. (2026b), who reach the same conclusion by a different route, measuring SonarQube issues introduced by merged bug-fix pull requests and finding that merge success does not reliably reflect post-merge quality. Two studies, different quality constructs, different corpora, the same result. It also carries a methodological consequence: for these signals a merged-only corpus is not a biased sample of agent output, which could not have been established without measuring the requests that were turned down.

RQ2, Can SOLID be measured at scale, and how faithfully? It can, with an operating characteristic that must be stated rather than collapsed into one number, and the two halves of that characteristic point in different directions. Precision divides sharply: Dependency Inversion reaches 0.890, and the other four sit between 0.330 and 0.420 as originally composed. Recall, once each stratum is weighted back to its size in the corpus, divides just as sharply: 0.733 to 1.000 for SRP, OCP and DIP, whose flag rates can therefore be read as upper bounds, but 0.352 for Liskov and 0.552 for Interface Segregation, whose cannot (Table 4.8).

Two of the four can be repaired, and the labels themselves say how. Restricting the Liskov flag to refused bequest, a base method that does real work, replaced by an override that only throws, gives precision 1.000 on 64 flags across two independent samples. Requiring an interface to carry at least 30 members raises Interface Segregation to 0.786. Both are restrictions rather than new signals, and both were derived from reading false positives rather than from tuning against the score.

Because both were fitted to a sample, both were re-scored on a holdout of 200 further entities per principle, drawn with a different seed and excluding every previously judged row. The Liskov restriction held exactly, 33 of 33; the width condition fell from 0.862 to 0.704, outside its own holdout interval. A categorical rule survives being fitted and a numeric threshold does not, which is the more general lesson of the exercise and the reason the pooled figures are the ones quoted here.

The gap that remains is the substantive answer. A structural proxy asks a syntactic question and the principles are semantic, and Open/Closed shows why that gap is not always closable: precision is flat across every dispatch width, because whether a set of

variants can grow is not a property of how many branches it currently has.

A second axis answers the objection that a judge reading the same source can be wrong in the same direction as the rubric. Three published analysers, run at their own defaults over 111,029 of the same production types, place the class-level instrument *inside* the disagreement they already show among themselves: our rubric agrees with PMD at $\kappa = 0.574$ and with DesigniteJava at $\kappa = 0.531$, while those two agree with each other at $\kappa = 0.384$ (Table 4.13). Two thirds of our flags are corroborated by at least one tool that knows nothing about this study. Where the tools diverge, the cause is the unit rather than the verdict: DesigniteJava charges the supertype for a substitutability fault where we charge the subclass, and works at package granularity for cycles where we work at compilation-unit granularity.

RQ3, Where does risk concentrate? Inside single units. Of roughly 4,100 single-responsibility violations and 270 substitutability violations present in the Java code these agents touched, the agents introduced a violation in two to four per cent of their pull requests (Table 4.18). The three relation-level principles show it most sharply: 681 refused-bequest relationships exist in the touched code, and pull requests introduced 16.

The explanation is structural. Single Responsibility and Open/Closed are properties of one unit, which a single pull request can author whole. Liskov, Interface Segregation and Dependency Inversion are properties of a relation between units, a base and its override, an interface and its implementers, a cycle among modules, and authoring both ends of such a relation in one contribution is rare. The consequence for practice is that per-change review can plausibly catch intra-unit damage and structurally cannot catch inter-unit damage, because the evidence for the latter is not present in any single diff.

5.3 Contributions

C1. A **validated, repository-scale measurement instrument for all five SOLID principles**, with every proxy's precision and recall reported against an independent reading judge rather than assumed. No prior study measures SOLID conformance, as opposed to detecting it in isolated examples, or measuring smells adjacent to it, on code that autonomous agents actually authored for real projects.

C2. A **corpus built on all three pull request outcomes**, which makes the review filter measurable instead of invisible, and the finding that for these signals the filter does not act. This is the direct answer to the survivorship-bias objection that a merged-only design cannot address from within.

C3. An **explicit map of where structural measurement of SOLID works and where it only screens**, including two detector refinements derived from labelled

false positives and a principled account of why the same kind of refinement is unavailable for Open/Closed. The map is falsifiable and re-testable, which a single pooled accuracy figure would not be.

C4. An **empirical account of where agent-authored design risk concentrates**: inside single units, not across them, measured in a language chosen specifically to remove the standing objection that SOLID is an awkward fit for languages which do not mandate object orientation.

C5. A **result about language semantics and measurability**: three of the five signals used to detect Liskov violations in a dynamically typed language cannot occur in Java code that compiles. What a type system enforces determines which violations a program can express, which is a methodological result independent of SOLID.

C6. An **auditable validation procedure**. Every label is required to trace back to the source the judge was shown, every label carries its recorded reason, and a mechanical audit enforces both. This makes the most subjective step in the study checkable by someone who does not trust the person who performed it.

5.4 Practical Implications

For a team choosing between coding agents, the SOLID profile in this corpus is a second axis alongside functional pass-rate benchmarks rather than a replacement for one. The differences between agents are real but small, and the more useful observation is the one that holds across all three: agent-authored classes are flagged at roughly half the rate of the pre-existing classes those agents edit. A reader who assumes agent output is the source of the structural problems in these repositories has the direction wrong.

For a team building an automated review gate, the results are directly actionable in one respect and directly cautionary in another. The single check worth adding today is **refused bequest**: a subclass overriding a working inherited method with a body that only throws. It is cheap to compute, it validated at 1.000 precision on 31 flags here and 0.823 on 62 flags in an earlier round, and it corresponds to something a reviewer would object to if they saw it. Interface width above roughly 30 members is a second usable signal. Conversely, a Single Responsibility or Open/Closed flag from a detector built this way should open a conversation rather than block a merge, since two thirds of such flags did not survive reading.

The broader implication follows from RQ1 and RQ3 together. Review does not appear to select on these properties, and the violations that matter most accumulate across changes rather than arriving in one. An automated structural check is therefore most valu-

able where human review is structurally unable to look: across the history of a component, not inside a single diff.

5.5 Limitations

The following bound every claim made above.

The labels are model-produced, not human ground truth. Every precision and recall figure is agreement between a deterministic rubric and a large language model reading the source. The judge can be wrong, and can be wrong in the same direction as the rubric. Two things constrain this. A second, independent reading of 174 overlapping entities agreed with the first 89.7% of the time, which places a ceiling on how precisely any agreement figure can be known. And the class-level instrument is compared against three independently developed analysers, which cannot share the study's biases by construction; it agrees with each of them more closely than they agree with each other (Section 4.7.2). That defence covers the single-responsibility instrument, which is the one the external tools also measure. Open/Closed and Interface Segregation have no counterpart in any published source-only Java analyser, so for those two the reading judge is the only validation available. The figures should be read as rubric-versus-judge agreement throughout, never as accuracy against ground truth.

Change size is a partial confound on the agent comparison. The agents write changes of very different sizes, and flag rates rise with size. Section 4.7.3 reports the comparison within size bands for this reason, and the pooled per-agent table should not be read on its own.

Agent and era are confounded. The median pull request date differs by seven months across the three agents. An agent measured on older code faces different repositories, framework versions and tasks. The per-agent ordering should be read with that in mind, and no claim is made that it would survive a period-matched sample.

"Rejected" is not "refused on quality". Most closed pull requests were never reviewed, which in practice means superseded, stale or abandoned. Only the `changes_requested` and `reviewed_then_closed` strata carry a recorded judgement, and they are small. The outcome contrast is reported by stratum for this reason, and the conclusion drawn from it is the narrow one: review does not appear to select on SOLID adherence.

Reconstructability differs by outcome. 62.1% of merged requests are analysable against 56.8% of rejected ones, and the reconstruction methods available differ between

them. The outcome comparison is therefore made only within a matched tier. Without that control it would partly be a comparison of which pull requests could be reconstructed rather than of what they contain.

Symbol resolution is incomplete, and unevenly so. Median supertype resolution is 87.0%, but 718 pull requests resolve below 50%, concentrated in framework-heavy projects. Liskov and Interface Segregation under-count there. The direction is safe, unresolved supertypes cause missed violations rather than invented ones, so the affected rates are lower bounds.

A single snapshot per pull request. The design compares two trees. Slow architectural erosion across many changes is invisible to it, which is precisely the process the intra-unit finding suggests matters most for the relation-level principles.

One language, and one dataset. The results are Java, drawn from AIDev. The framework is language-parameterised and a second language is an additional fact extractor rather than a second study, but nothing here demonstrates that the per-agent ordering transfers. A benchmark defined over specific pull requests also degrades as the ecosystem moves: 18 requests were already unrecoverable because repositories had been deleted or their history rewritten.

Small absolute counts on the relation-level principles. Sixteen pull requests introduce a Liskov violation and 195 a dependency cycle. These are reported as counts beside every rate so that a reader can weigh them directly rather than trust a percentage a handful of pull requests could move.

5.6 Future Work

- **A churn-based signal for Open/Closed.** The finding that openness is uncorrelated with dispatch width removes the obvious threshold and suggests the replacement: a class edited across many pull requests is, by definition, not closed for modification. The corpus already carries the change history required to compute it.
- **A closed-set suppressor that recognises external specifications.** The existing suppressor recognises language builtins and fired on none of the flagged rows. The closed sets that actually occur are JDBC type codes, protobuf scalars, Unicode blocks and generated parser tables. Recognising specification-derived enumerations is a concrete and testable improvement.

- **Recall for the two blind detectors.** The weighted recall of the Liskov and Interface Segregation proxies rests on the few violations found in an unflagged stratum that is fifty times larger than the flagged one, which makes their intervals wide. A sample drawn only from unflagged entities, sized for that question alone, would narrow them; the three misses already found say where to look, at silent refusals rather than declared ones.
- **A second language with the same agent coverage**, to test whether the per-agent ordering is a property of the agents or of how they write Java. The pipeline shape ports; the obstacle is corpus availability rather than method.
- **A repository-scale unit of analysis.** The intra-unit finding implies that inter-unit damage accumulates outside the reach of any per-change measurement. Measuring a component across its history, rather than a diff, is the natural next design.

5.7 Closing Remark

The central result of this thesis is a distinction rather than a verdict. The design faults that autonomous coding agents leave in real code are concentrated inside single units, a class doing too much, a conditional that should have been an extension point, and are largely absent from the relations between units that one change rarely gets to author in full. Most of what the instruments find in these repositories was written by people, before the agent arrived.

That distinction held across three agents, across merged and rejected work alike, and in the language most likely to break it. What remains open is whether it holds as the unit of measurement grows: a repository maintained over months, by an agent and by the humans who edit its output afterwards, is the scale at which architecture is actually won or lost, and it is not the scale at which a pull request can be read.

Bibliography

Anonymous (under review) (2026). Beyond functional correctness: Design issues in ai ide-generated large-scale projects. arXiv:2604.06373.

Anthropic (2025). Effective context engineering for ai agents. Vendor engineering report.

Ben Sghaier, O., Li, H., Adams, B., and Hassan, A. E. (2026). Don't blame the large language model: How agent harness evolution shapes coding agent quality. arXiv:2607.03691.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Cynthia, S. T., Das, J. K., and Roy, B. (2026a). Are we all using agents the same way? an empirical study of core and peripheral developers' use of coding agents. arXiv:2601.20106.

Cynthia, S. T., Muttakin, A., and Roy, B. (2026b). Beyond bug fixes: An empirical investigation of post-merge code quality issues in agent-generated pull requests. arXiv:2601.20109.

et al., L. (2026). Realbench: A repo-level code generation benchmark aligned with real-world software development practices. arXiv:2604.22659.

Fowler, M., Beck, K., Brant, J., Opdyke, W., and Roberts, D. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.

Fsoft AI Center (2025). Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470.

Henderson-Sellers, B. (1996). *Object-Oriented Metrics: Measures of Complexity*. Prentice Hall. Source of the LCOM cohesion measures, including the LCOM4 connected-components formulation.

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., and Cui, H. (2023). Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. arXiv:2312.13010.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. (2024). Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv:2403.07974.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*.

Konrad, P. M., Adam, T. L., Terrenzi, R., and Ayvaz, S. (2026). Architecture without architects: How ai coding agents shape software architecture. arXiv:2604.04990.

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 3rd edition.

Li, H., Zhang, H., and Hassan, A. E. (2025). The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. arXiv:2507.15003; AIDev dataset, replication package at github.com/SAILResearch/AI_Teammates_in_SE3 and huggingface.co/datasets/hao-li/AIDev.

Lu, P., Zhang, S., Hou, Y., Ye, L., Huang, C., Chen, Z., Zeng, J., Jiang, H., Liu, P., Wang, Y., and Yang, M.-H. (2026). Projdevbench: Benchmarking ai coding agents on end-to-end project development. arXiv:2602.01655.

Marcus, A., Poshyvanyk, D., and Ferenc, R. (2008). Using the conceptual cohesion of classes for fault prediction in object-oriented systems. *IEEE Transactions on Software Engineering*, 34(2):287–300. Origin of the C3 conceptual-cohesion metric used as the SRP semantic suppressor.

Martin, R. C. (2000). *Design Principles and Design Patterns*. Foundational reference for the SOLID principles.

Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. Source of the package metrics used for DIP: Instability (I), Abstractness (A), and Distance from the Main Sequence (D = |A + I - 1|).

Mazloomzadeh, I., Morovati, M. M., and Khomh, F. (2026). How do AI coding agents contribute to software development? an empirical study of agentic pull requests. arXiv:2607.21832.

METR (2026). Many swe-bench-passing prs would not be merged into main. METR research note, 10 March 2026.

Pehlivan, F., Ergüzen, A. U., Moslemi Yengejeh, S., Lami, M., and Koyuncu, A. (2025). Are we solid yet? an empirical study on prompting llms to detect design principle violations. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2509.03093; 240 manually validated examples.

Sawada, S., Shirai, T., Kashiwa, Y., Yamaguchi, K., Iwata, H., and Iida, H. (2026). To what extent does agent-generated code require maintenance? an empirical study. In *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. arXiv:2605.06464; built on the same AIDev pull-request dataset used in this thesis.

SprocketLab (2026). Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160. Strongly-connected-components algorithm used for DIP import-cycle detection.

Xu, F., Medappa, P. K., Tunc, M. M., Vroegindeweij, M., and Fransoo, J. C. (2025). Ai-assisted programming decreases the productivity of experienced developers by increasing the technical debt and maintenance burden. arXiv:2510.10165.

Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.05685; methodological basis and caveats for using an LLM as the validation label judge.

Zhu, H., Zhang, Q., Wang, J., Yang, Z., and Qiu, Y. (2026a). Needle in the repo: A benchmark for maintainability in ai-generated repository edits. arXiv:2603.27745.

Zhu, Y., Tsantalis, N., and Rigby, P. C. (2026b). Ai-generated smells: An analysis of code and architecture in llm and agent-driven development. arXiv:2605.02741.

Zhuo, T. Y. et al. (2025). Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.15877 (2024); 1,140 tasks over 139 libraries.

Chapter

A

Pipeline Run Order and Reproducibility

This appendix lists the exact stage order used to reproduce the results reported in Chapter 4. Every stage is resumable and checkpointed; re-running a stage after a threshold change requires deleting that stage's output parquet first, since resumption otherwise keeps previously-computed rows.

```
curate.py --lang java      -> gold set        6,956 PRs / 1,010 repos,
                                              balanced on agent x outcome
clone_and_extract.py       -> file lists      GitHub API where local
                                              history is insufficient
mvn -o package (sidecar)   -> facts jar       JavaParser + SymbolSolver
run_java.py                -> five parquets   one parsed tree feeds all
                                              five principles; 4,245 ok
   (thresholds calibrated from corpus percentiles, then parquets
    deleted and the pass re-run, so no flag predates its threshold)
report_java.py             -> per-agent and per-outcome summaries
make_java_labeling_sample.py  -> 100 flagged + 100 unflagged per principle
extract_java_label_batches.py -> the source shown to the judge
audit_label_batches.py     -> GATE: every row must trace to that source,
                              and no batch may leak the detector's verdict
score_labels.py            -> precision and recall per principle,
                              per agent, per outcome
run_crosscheck_java.py     -> CK, PMD, DesigniteJava over the same trees
verify_v2_java.py          -> whole-corpus invariants; exits non-zero
                              on any failure
```

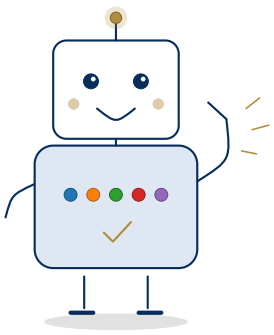
A single combined delta pass (`run_java.py`) computes all five principles from one parsed tree, since parsing costs 10–20 seconds per tree and five separate harnesses would each pay that cost independently. Extracted facts are cached by the content hash of the tree, so re-running cannot silently change an input.

Software and Data Availability

The corpus is derived from the public AIDev dataset (Li et al., 2025). The analysis pipeline, configuration, and result tables are version-controlled; source code is available from the author upon request. No proprietary code, personal data, or human-participant data was collected or is distributed as part of this work.

Configuration Discipline

All thresholds, file paths, and policy knobs (agent PR caps, star and license floors, rubric cut-points, suppressor toggles) live in a single, heavily-commented configuration module per language, so that a threshold change is a one-line, auditable edit rather than a scattered one. Every pure evaluator module carries an inline self-check, a small hand-built example with a known answer, run whenever that module changes, which is the closest analogue this pipeline has to a unit-test suite, chosen deliberately over a conventional test framework so that the check travels with the module it verifies.



if you made it this far, thank you for reading, built one commit, one validated proxy, at a time.